



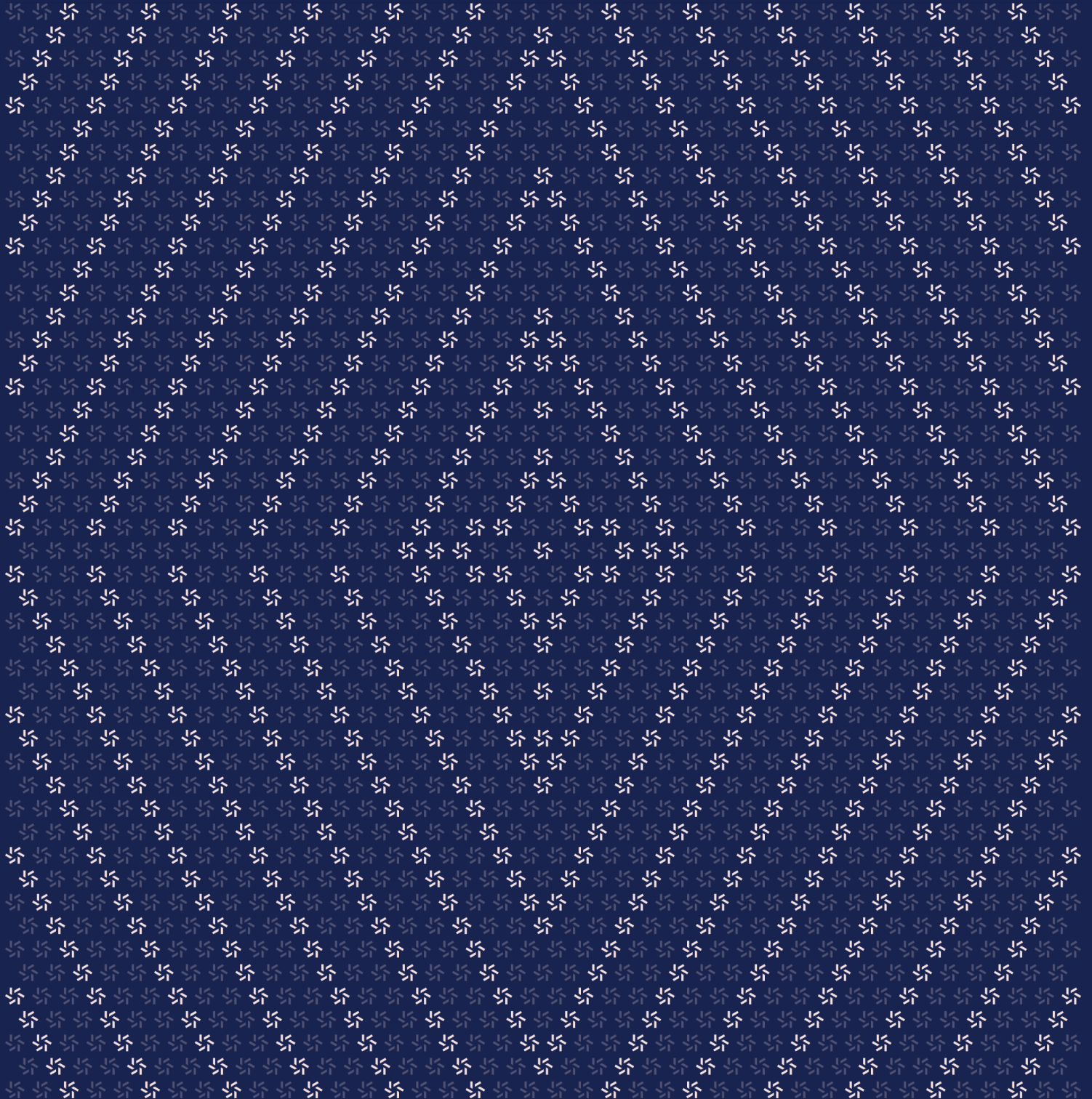
June 29, 2026

Prepared for
Rajath
Ananth
Shanti
Aditya
Zer0dot
Polymarket

Prepared by
Katerina Belotskaia
Kuilin Li
Zellic

Polymarket V2

Smart Contract Security Assessment



Contents

About Zellic	5
1. Overview	6
1.1. Executive Summary	6
1.2. Goals of the Assessment	6
1.3. Non-goals and Limitations	6
1.4. Results	7
2. Introduction	8
2.1. About Polymarket V2	8
2.2. Methodology	8
2.3. Scope	10
2.4. Project Overview	12
2.5. Project Timeline	12
3. Detailed Findings	13
3.1. <code>initializeRequest</code> allows misconfiguration that prevents market resolution	13
3.2. A deferred reward refund in <code>OptimisticOracleModule</code> is unrecoverable and permanently blocks disputes for the affected condition	16
3.3. Skipped zeroing of higher-order bits in assembly types causes varied issues, including the emission of malformed events	19
3.4. A single disputer module can satisfy <code>disputerThreshold</code> by calling <code>disputeResult</code> repeatedly	22
3.5. Module registry lacks a reverse address mapping, allowing duplicate registrations and incorrect <code>crossModuleAuth</code> clearing	24

3.6.	<code>priceDisputed</code> lacks the resolved-status guard present in <code>priceSettled</code> , allowing a post-settlement callback to trigger a spurious re-request	27
3.7.	Auto-resolution of the synthetic Other condition bypasses <code>resolutionPausedAt</code> for the fallback condition ID	29
3.8.	An empty <code>_positionIds</code> array bypasses the <code>onlyModuleByPositionIds</code> authorization check, allowing any caller to emit spurious <code>TransferBatch</code> events via <code>batchMint</code> and <code>batchBurn</code>	31
3.9.	Modules with <code>crossModuleAuth</code> can split and merge positions belonging to other modules	33
3.10.	<code>PositionIdLib.encode</code> implicitly assumes <code>ResolutionChain.POLYGON</code> and is named inconsistently with its counterparts	35
3.11.	<code>_processMessage</code> silently corrupts the caller's <code>_message</code> reference via an unaligned in-place slice; <code>_trimArray</code> similarly invalidates its caller's array through aliasing	38
3.12.	Splits that produce zero-value tokens can theoretically be grieved by frontrunners	41
3.13.	Modules unconditionally accept incoming ERC1155 transfers but provide no recovery path for positions with a mismatched module ID	43
4.	Discussion	45
4.1.	<code>ResolutionStatus.Active</code> is never written to storage	45
4.2.	<code>ManagedOptimisticOracleV2</code> extends the dispute window past liveness expiry by masking <code>Expired</code> requests as <code>Proposed</code>	45
4.3.	<code>ManagedOptimisticOracleV2</code> validates liveness against <code>minimumDisputeWindow</code> , which can cause <code>setCustomLiveness</code> to revert if the window is raised	46
4.4.	Absence of the <code>crossModuleAuth</code> flag does not meaningfully restrict module capabilities	47
4.5.	Upgrade checks overstate immutability guarantees	47
4.6.	<code>CombinatorialModule</code> does not need <code>crossModuleAuth</code> flag if it retains child positions to back its positions	48

5.	Threat Model	50
5.1.	Module: BridgeBase.sol	50
5.2.	Module: CcipBridge.sol	57
5.3.	Module: ChainlinkReporterModule.sol	61
5.4.	Module: CombinatorialModule.sol	65
5.5.	Module: OracleAggregator.sol	99

6.	Assessment Results	111
6.1.	Disclaimer	111

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) [worldwide](#) in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io [and follow @zellic_io](#) [on Twitter](#). If you are interested in partnering with Zellic, contact us at hello@zellic.io [.](#)



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Polymarket from June 1st to June 26th, 2026. During this engagement, Zellic reviewed Polymarket V2's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Is there any way to resolve a legacy market and compromise the integrity of the migrated market?
 - Is there a way to bridge positions in such a way as to receive more than one bridged position?
 - Is there a way to compromise invariants (i.e., horizontal split/merge equivalence) via bridging?
 - Is there any concern with inheritance chains in any upgradable contract that might come back to bite us in the future?
 - Are there any caveats to the removal of “condition” and “event” preparation, particularly when it comes to bridging?
 - Are the differences between the codebase's collateralToken and ramps versus the deployed ones problematic in some way?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

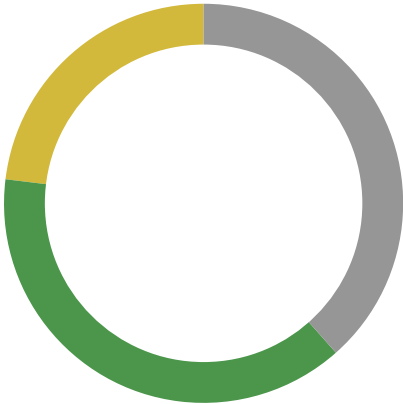
1.4. Results

During our assessment on the scoped Polymarket V2 contracts, we discovered 13 findings. No critical issues were found. Three findings were of medium impact, five were of low impact, and the remaining findings were informational in nature.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Polymarket in the Discussion section (4.7).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	3
Low	5
Informational	5



2. Introduction

2.1. About Polymarket V2

Polymarket contributed the following description of Polymarket V2:

Polymarket V2 is the next evolution of the Polymarket on-chain prediction market protocol. It includes numerous new features, hardened security constraints, and efficiencies.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood.

We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped contracts itself. These observations — found in the Discussion ([4.7](#)) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

Polymarket V2 Contracts

Type	Solidity
Platform	EVM-compatible

Target	polymarket-v2
Repository	https://github.com/Polymarket/polymarket-v2 ↗
Version	c3be835d5d6febad4e52e43739835a2adc071fc8
Programs	positionManager/PositionManager.sol modules/CombinatorialModule.sol modules/NegRiskModule.sol modules/BinaryModule.sol modules/abstract/BaseModule.sol modules/abstract/ModuleErrors.sol modules/abstract/OracleModule.sol modules/migration/BaseMigrationMixin.sol modules/migration/BinaryMigrationMixin.sol modules/migration/NegRiskMigrationMixin.sol oracle/OracleAggregator.sol oracle/modules/OptimisticOracleModule.sol oracle/modules/reporters/ChainlinkReporterModule.sol oracle/modules/reporters/EOAReporterModule.sol oracle/abstract/OracleModuleBase.sol oracle/abstract/OracleAggregatorErrors.sol oracle/abstract/OracleAggregatorEvents.sol oracle/mixins/Auth.sol oracle/mixins/Pausable.sol oracle/libraries/OptimisticOraclePayoutLib.sol collateral/CollateralToken.sol collateral/PermissionedRamp.sol collateral/CollateralOnramp.sol collateral/CollateralOfframp.sol collateral/abstract/Pausable.sol collateral/abstract/CollateralErrors.sol exchange/Exchange.sol exchange/OrderStructs.sol routers/Router.sol routers/CtfRouter.sol routers/BridgeRouter.sol bridge/abstract/BridgeBase.sol bridge/CcipBridge.sol libraries/Ids.sol libraries/BridgePayloads.sol libraries/CrossChainTypes.sol libraries/ModuleIds.sol auth/Roles.sol auth/InitializableRoles.sol utils/AutoRedeemer.sol abstract/ERC1155TokenReceiver.sol

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 5.4 person-weeks. The assessment was conducted by two consultants over the course of 4 calendar weeks.

Contact Information

The following project manager was associated with the engagement:

Pedro Moura
✈ Engagement Manager
pedro@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Katerina Belotskaia
✈ Engineer
kate@zellic.io ↗

Kuilin Li
✈ Engineer
kuilin@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

June 2, 2026 Kick-off call

June 1, 2026 Start of primary review period

June 26, 2026 End of primary review period

3. Detailed Findings

3.1. initializeRequest allows misconfiguration that prevents market resolution

Target	OracleAggregator		
Category	Coding Mistakes	Severity	Medium
Likelihood	Low	Impact	Medium

Description

The `initializeRequest` function in `OracleAggregator` enforces only a lower-bound check on `reporterThreshold` and only a nonzero check on `targetContract`, allowing two classes of misconfiguration that are both permanent once the event is registered.

The first class concerns `reporterThreshold`. The function requires the threshold to be at least one, but performs no check against the number of registered reporter modules:

```
require(_params.reporterThreshold >= 1, InvalidConfig());
```

Because `hasReporterVoted` prevents any module from casting more than one vote per request, a `reporterThreshold` set above the length of `reporterModules` means the vote count for any proposed result hash can never reach the threshold. The promotion check in `reportResult` is therefore unreachable:

```
if (newCount >= cfg.reporterThreshold && state.proposedResultHash ==
    bytes32(0)) {
    state.proposedResultHash = resultHash;
    state.disputeWindowEnd = uint40(block.timestamp + cfg.livenessWindow);
    // [...]
}
```

With no proposal hash ever set, neither `finalize` nor the liveness path can proceed. The market remains perpetually `Active`. However, the admin `resolveResult` function bypasses the reporter and disputer pipeline entirely, allowing an admin to resolve the market directly without the threshold ever being reached.

The second class concerns `targetContract`. The function requires only that the address be nonzero, with no check that it is a live module registered in `PositionManager`:

```
require(_params.targetContract != address(0), InvalidConfig());
cfg.targetContract = _params.targetContract;
```

Every resolution path, including the admin `resolveResult` override, ultimately reaches `_finalizeConditions`, which calls `reportResult` on the stored address:

```
try IBinaryReporter(_cfg.targetContract).reportResult(reportConditionId,
    binaryResult) { }
catch (bytes memory returnData) {
    // [...]
    assembly {
        revert(add(returnData, 32), mload(returnData))
    }
}
```

A wrong address makes this call meaningless or causes it to revert.

Impact

A misconfigured `reporterThreshold` produces a market that is stuck Active indefinitely, but the admin `resolveResult` override remains functional and provides a recovery path. In the case of a misconfigured `targetContract`, even the emergency resolution path is broken: the market is permanently unresolvable, and positions held by users in it cannot be redeemed.

Recommendations

We recommend adding an upper-bound check on `reporterThreshold` to ensure the registered module set can satisfy it:

```
require(_params.targetContract != address(0), InvalidConfig());
require(_params.reporterThreshold <= _params.reporterModules.length,
    InvalidConfig());
require(_params.reporterThreshold >= 1, InvalidConfig());
```

We also recommend storing a reference to `PositionManager` in `OracleAggregator` and validating `targetContract` against the module registry at initialization time, so only live registered modules can be set as resolution targets:

```
uint256 modId = IPositionManagerModule(_params.targetContract).moduleId();
require(POSITION_MANAGER.moduleById(modId) == _params.targetContract,
    InvalidConfig());
```

Remediation

This issue has been acknowledged by Polymarket, and a fix was implemented in commit [e8078260](#).

3.2. A deferred reward refund in OptimisticOracleModule is unrecoverable and permanently blocks disputes for the affected condition

Target	OptimisticOracleModule		
Category	Coding Mistakes	Severity	Medium
Likelihood	Low	Impact	Medium

Description

OptimisticOracleModule calls `setEventBased` on every request and re-request via `_setupRequest`:

```
optimisticOracle.setEventBased(_identifier, _timestamp, _ancillaryData);
```

`setEventBased` sets `refundOnDispute = true` on the underlying OOV2 request. When a dispute occurs, `disputePriceFor` in the MOOV2-forked `OptimisticOracleV2` refunds the reward to the requester (the module) before invoking the callback:

```
if (request.reward > 0 && request.requestSettings.refundOnDispute) {
    refund = request.reward;
    request.reward = 0;
    _transferOrDeferPayout(request.currency, requester, refund);
}
// ... then invokes requester.priceDisputed(...)
```

`_transferOrDeferPayout` uses `trySafeTransfer`. If that call reverts or returns false the amount is not transferred to the module. Instead it is recorded in the OOV2's `deferredPayouts` mapping:

```
function _transferOrDeferPayout(IERC20 currency, address recipient,
    uint256 amount) private {
    if (!currency.trySafeTransfer(recipient, amount)) {
        deferredPayouts[currency][recipient] += amount;
        emit PayoutDeferred(address(currency), recipient, amount);
    }
}
```

`deferredPayouts` is claimed via `claimDeferredPayout`, which requires `msg.sender` to be the deferred recipient:


```
function claimDeferredPayout(IERC20 currency, address repaymentAddress)
    external override nonReentrant {
        address deferredRecipient = msg.sender; // must be the module
        uint256 amount = deferredPayouts[currency][deferredRecipient];
        // ...
    }
```

OptimisticOracleModule has no function that calls `claimDeferredPayout`, so any deferred refund is permanently stuck in the OOV2's storage with no way to retrieve it.

The deferred refund also triggers a secondary failure. After `_transferOrDeferPayout` returns (without sending tokens to the module), the OOV2 proceeds to call `priceDisputed` on the module. `priceDisputed` calls `_reRequest`, which calls `_setupRequest`, which calls `requestPrice` on the OOV2 with reward: `currentReward[conditionId]`. `requestPrice` pulls the reward from the module via `safeTransferFrom(module, oov2, reward)`. Because the refund was deferred instead of transferred, the module holds no balance for that token and the pull reverts. The revert propagates through `_reRequest` -> `priceDisputed` -> back to the OOV2's `disputePriceFor` as the callback returns with a revert.

The full `disputePrice` transaction therefore reverts. Every subsequent attempt to dispute any proposal for that condition will follow the same path and revert for the same reason: the module's balance remains zero and `requestPrice` cannot pull the reward. Disputes for that condition are permanently blocked until an external actor transfers the reward amount directly to the module, at which point the already-deferred amount in OOV2 is still unrecoverable without a `claimDeferredPayout` wrapper.

Impact

If the deferred-payout path is triggered, two harms follow. First, the reward tokens sent by the original `createRequest` caller are permanently locked in the OOV2's `deferredPayouts` storage with no on-chain mechanism to recover them, since `OptimisticOracleModule` has no `claimDeferredPayout` wrapper. Second, disputes for the condition are blocked: every `disputePrice` call reverts until someone manually transfers the reward amount directly to the module's address. Once the module's balance is restored that way, disputes resume, but the deferred amount in OOV2 remains irrecoverable and represents a separate, permanent loss.

The low likelihood reflects that `trySafeTransfer` fails only in edge cases specific to the bond token (blacklisting, fee-on-transfer behavior, or similar restrictions).

Recommendations

We recommend adding a wrapper function to `OptimisticOracleModule` that forwards the `claimDeferredPayout` call to the OOV2, allowing an operator to recover a deferred refund and restore the module's balance before the next re-request attempt:

```
function claimDeferredPayout(address _currency, address _repaymentAddress)
    external onlyOperator {

    IOptimisticOracleV2(optimisticOracle).claimDeferredPayout(IERC20(_currency),
        _repaymentAddress);
}
```

Remediation

This issue has been acknowledged by Polymarket. Polymarket stated that the OptimisticOracleModule has been removed in favor of the new OOReporter added in [e8078260](#).

3.3. Skipped zeroing of higher-order bits in assembly types causes varied issues, including the emission of malformed events

Target	PositionManager, Exchange		
Category	Coding Mistakes	Severity	Medium
Likelihood	Low	Impact	Medium

Description

According to [the Solidity documentation](#) regarding external variable access in YUL assembly:

If you access variables of a type that spans less than 256 bits (for example uint64, address, or bytes16), you cannot make any assumptions about bits not part of the encoding of the type. Especially, do not assume them to be zero. To be safe, always clear the data properly before you use it in a context where this is important.

In the PositionManager contract's `mint` function, the address type, which is 160 bits long, is passed through the calldata directly to the emission of an event:

```
function mint(address _to, PositionId _positionId, uint256 _amount)
    external onlyModuleByPositionId(_positionId) {
    assembly ("memory-safe") {
        // [... code that does not change _to]
        log4(0x00, 0x40,
            0xc3d58168c5ae7397731d063d5bbf3d657854427343f4c083240f7aacaa2d0f62,
            caller(), 0, _to)
    }
}
```

This means that if an attacker can call the `mint` function in a nonstandard way to make the higher-order bits of `_to` nonzero, the event will be emitted with those dirty bits, which causes the event to fail to decode as actual events in most off-chain EVM libraries.

The same issue, where inline YUL assembly assumes that the higher-order bits of an address field are cleared by the Solidity compiler, is present in the following functions:

- `PositionManager.mint`
- `PositionManager.batchMint`
- `Exchange._emitTakerEvents`
- `Exchange._emitOrderFilled`

- `Exchange._safeSalt`
- `Exchange._computeStructHash`

Impact

The impact of this issue is dependent on implementation details of the specific Solidity compiler version used to compile this code. The impact documented here was analyzed under a specific Solidity version communicated to Polymarket, but other versions and compilers can potentially have arbitrary other impacts.

For the `PositionManager` functions `mint` and `batchMint`, there is no impact. Because the address `_to` is taken directly from the calldata, which is ABI-encoded and compacted, there is no opportunity for an attacker to cause the higher bits to be dirty without causing the function to revert.

For the `Exchange` functions `_emitTakerEvents` and `_emitOrderFilled`, instead of being directly in the calldata, the address is taken from a struct that includes an address field. So, an attacker does have the opportunity to dirty those bits, and the Solidity compiler does not check that those bits are cleared at the entrypoint. This results in the `OrderFilled` and `OrdersMatched` events being emitted with dirty bits in their address fields, which then causes most EVM libraries to fail to decode the event into a valid higher-abstraction-level event for further processing. So, an attacker can cause off-chain code to skip potentially-important logic.

For the `Exchange` functions `_safeSalt` and `_computeStructHash`, the address is also taken from a struct in the calldata, so an attacker can still dirty those bits. Since these functions hash the struct, this will cause the order to have a different hash on-chain than a naive off-chain hash would output. The impact of this depends on the off-chain components of this project.

Recommendations

We recommend clearing the higher-order bits in the way the Solidity documentation recommends. For instance, for the `PositionManager` `mint` function:

```
function mint(address _to, PositionId _positionId, uint256 _amount)
    external onlyModuleByPositionId(_positionId) {
    assembly ("memory-safe") {
        // [... code that does not change _to]

        log4(0x00, 0x40, 0xc3d58168c5ae7397731d063d5bbf3d657854427343f4c083240
            f7aaca2d0f62, caller(), 0, _to)

        log4(0x00, 0x40, 0xc3d58168c5ae7397731d063d5bbf3d657854427343f4c083240
            f7aaca2d0f62, caller(), 0,
            and(0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF, _to))
```

```
    )  
  }  
}
```

Remediation

This issue has been acknowledged by Polymarket, and a fix was implemented in commit [3ce12a45](#).

3.4. A single disputer module can satisfy `disputerThreshold` by calling `disputeResult` repeatedly

Target	OracleAggregator		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

The `reportResult` function prevents any reporter module from casting more than one vote per request by recording each caller in `hasReporterVoted` and reverting on a repeated call:

```
require(!hasReporterVoted[_requestId][msg.sender], AlreadyVoted());
hasReporterVoted[_requestId][msg.sender] = true;
```

The `disputeResult` function applies no equivalent guard. Any registered disputer module can call it any number of times on the same request, and each call increments `disputeCount` unconditionally:

```
require(isDisputerModule[eventId][msg.sender], NotRegisteredModule());

state.disputeCount++;

// [...]

if (state.disputeCount >= cfg.disputerThreshold) {
    state.status = ResolutionStatus.ArbitrationRequested;
    // [...]
}
```

Because `disputeCount` is not bounded to one increment per module, a single registered disputer can reach any `disputerThreshold` by calling `disputeResult` that many times in succession, triggering arbitration without any other disputer participating. This defeats the purpose of setting a threshold above one.

Impact

A disputer module that is compromised or acts outside its intended role can unilaterally escalate any proposed outcome to arbitration, stalling finalization and forcing the arbitrator pipeline to

engage regardless of whether a genuine dispute exists. Because disputer modules are operator-registered trusted parties, this requires a breach of that trust, which keeps the likelihood low. The admin `resolveResult` override remains available to recover a market that has been improperly escalated.

Recommendations

We recommend adding a per-module, per-request vote record for disputers, mirroring the existing `hasReporterVoted` pattern:

```
mapping(bytes32 => mapping(address => bool)) public hasDisputerVoted;

function disputeResult(bytes32 _requestId) external whenUnpaused {
    // [...]
    require(!hasDisputerVoted[_requestId][msg.sender], AlreadyVoted());
    hasDisputerVoted[_requestId][msg.sender] = true;

    state.disputeCount++;
    // [...]
}
```

Remediation

This issue has been acknowledged by Polymarket, and a fix was implemented in commit [e8078260](#).

3.5. Module registry lacks a reverse address mapping, allowing duplicate registrations and incorrect crossModuleAuth clearing

Target	PositionManager		
Category	Code Maturity	Severity	Low
Likelihood	Low	Impact	Low

Description

The PositionManager module registry is keyed exclusively by module ID: addModule calls BaseModule(_module).moduleId() to obtain the self-reported identifier, then stores moduleById[moduleId] = _module. No reverse mapping from address to ID exists.

```
function addModule(address _module) external onlyAdmin {
    uint256 moduleId = BaseModule(_module).moduleId();
    require(moduleId != 0, InvalidModuleId());
    require(moduleById[moduleId] == address(0), ModuleAlreadyRegistered());

    moduleById[moduleId] = _module;
    // [...]
}
```

The ModuleAlreadyRegistered guard prevents registering a second address under an already-occupied ID, but it does not prevent registering the same address under a second ID. If a module contract reports a different ID under different circumstances, addModule would succeed and the address would occupy two slots in moduleById. Whether this is intentional to support module templating or similar patterns is not clear from the codebase.

Regardless of that intent, the absence of a reverse mapping creates a correctness issue in removeModule. When a module is deregistered, the function reads the address from the ID slot, then unconditionally clears crossModuleAuth for that address if it was set:

```
function removeModule(uint256 _moduleId) external onlyAdmin {
    address module = moduleById[_moduleId];
    require(module != address(0), ModuleNotRegistered());

    delete moduleById[_moduleId];
    if (crossModuleAuth[module]) {
        delete crossModuleAuth[module];
        emit CrossModuleAuthSet(module, false);
    }
}
```



```
// [...]  
}
```

This logic assumes a one-to-one relationship between module addresses and IDs. If the same address is registered under two IDs and `removeModule` is called for one of them, `crossModuleAuth` is revoked for the address even though it remains registered under the second ID. Subsequent mint and burn calls from that address, which rely on `crossModuleAuth` as a fallback authorization path, would then fail.

The missing reverse mapping also affects the external interface. Given only a module address, third parties and off-chain tooling must first call `moduleId()` on the module contract to learn its ID, then call `moduleById` on the `PositionManager` to confirm it is the registered address for that ID. A `moduleByAddress` reverse mapping would make this a single lookup and would allow `removeModule` to confirm that the address has no other registered IDs before clearing `crossModuleAuth`.

Impact

If the same module address is registered under two IDs and an admin removes one of them, `crossModuleAuth` is incorrectly cleared. The module loses its cross-module mint and burn authorization for the remaining ID, causing all operations that depend on `crossModuleAuth` to revert with `Unauthorized`. This requires the specific combination of a module reporting multiple IDs and an admin removing one registration, which keeps the likelihood low.

Recommendations

We recommend adding a reverse mapping from module address to module ID and using it in both `addModule` and `removeModule`:

```
mapping(uint256 => address) public moduleById;  
mapping(address => uint256) public moduleIdByAddress;
```

In `addModule`, reject a registration if the address is already present under any ID:

```
function addModule(address _module) external onlyAdmin {  
    uint256 moduleId = BaseModule(_module).moduleId();  
    require(moduleId != 0, InvalidModuleId());  
    require(moduleById[moduleId] == address(0), ModuleAlreadyRegistered());  
    require(moduleIdByAddress[_module] == 0, ModuleAlreadyRegistered());  
  
    moduleById[moduleId] = _module;  
    moduleIdByAddress[_module] = moduleId;  
    // [...]
```

```
}
```

In `removeModule`, use the reverse mapping to clear both entries atomically:

```
function removeModule(uint256 _moduleId) external onlyAdmin {
    address module = moduleById[_moduleId];
    require(module != address(0), ModuleNotRegistered());

    delete moduleById[_moduleId];
    delete moduleIdByAddress[module];
    if (crossModuleAuth[module]) {
        delete crossModuleAuth[module];
        emit CrossModuleAuthSet(module, false);
    }
    // [...]
}
```

Remediation

This issue has been acknowledged by Polymarket.

- 3.6. `priceDisputed` lacks the resolved-status guard present in `priceSettled`, allowing a post-settlement callback to trigger a spurious re-request

Target	OptimisticOracleModule		
Category	Code Maturity	Severity	Low
Likelihood	Low	Impact	Low

Description

`priceSettled` contains two staleness guards before it acts. The first checks that the incoming callback matches the most recent request ID; the second checks whether the aggregator has already resolved the condition (for example, via an admin override):

```
// Only act if this is the most recent request.
if (requestId != currentRequestId[conditionId]) return;

// If the aggregator already resolved (e.g. admin override), exit early.
(uint8 status,,, ) =
    IOracleAggregator(aggregator).getRequestState(bytes32(ConditionId.unwrap(conditionId)))
if (status == uint8(OracleAggregator.ResolutionStatus.Resolved)) return;
```

`priceDisputed` applies only the first check and has no equivalent resolved-status guard:

```
// Only act if this is the most recent request.
if (requestId != currentRequestId[conditionId]) return;

// No resolved-status check – proceeds directly to _reRequest.
_reRequest({ _conditionId: conditionId, ... });
```

After `priceSettled` successfully resolves a condition, `currentRequestId[conditionId]` is not cleared. It continues to hold the settled request ID. If the Optimistic Oracle were to call `priceDisputed` after that settlement — which is outside the normal OOV2 lifecycle — `priceDisputed` would find `requestId == currentRequestId[conditionId]`, skip the staleness guard, and proceed to `_reRequest`. A new OOV2 request would be registered under a fresh timestamp, `currentRequestId[conditionId]` would be overwritten, and reward and bond token flows would be initiated against the module's holdings, all for a condition that has already reached its terminal state.

The aggregator itself would not double-resolve: `resolveResult` in `OracleAggregator` returns early when the condition is already `Resolved`. The harm is confined to the module's internal lifecycle state and token movements.

Impact

A spurious priceDisputed callback on a settled request causes the module to open a new OOv2 price request, overwrite currentRequestId, and move reward and bond funds into the new request. The aggregator is protected against double-resolution, so no position payouts are affected, but the module's lifecycle state advances past its intended terminal point and any funds held for the settled condition are tied up in an unneeded request.

Recommendations

We recommend adding the same resolved-status guard to priceDisputed that already exists in priceSettled, so that any callback arriving after settlement is ignored at the module level:

```
function priceDisputed(bytes32 identifier, uint256 timestamp,
    bytes memory ancillaryData, uint256)
    external
    override
    onlyOptimisticOracle
{
    bytes32 requestId = _computeRequestId(identifier, timestamp,
    ancillaryData);
    ConditionId conditionId = requestToCondition[requestId];

    if (requestId != currentRequestId[conditionId]) return;

    (uint8 status,,, ) = IOracleAggregator(aggregator).getRequestState(bytes32(
        ConditionId.unwrap(conditionId)));
    if (status == uint8(OracleAggregator.ResolutionStatus.Resolved)) return;

    // [...]
}
```

Remediation

This issue has been acknowledged by Polymarket. Polymarket stated that the OptimisticOracleModule has been removed in favor of the new OOReporter added in [e8078260](#).

3.7. Auto-resolution of the synthetic Other condition bypasses resolution-PausedAt for the fallback condition ID

Target	NegRiskModule		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

The `onlyResolver` modifier enforces the per-condition resolution pause for the condition ID passed to `reportResult`:

```
modifier onlyResolver(ConditionId _id) {  
    // [...]  
    require(resolutionPausedAt[_id] == 0, ResolutionIsPaused());  
    _;  
}
```

When a real condition is reported, `reportResult` calls `_finalizeNegriskResolution`, which may trigger auto-resolution of the synthetic Other condition (the fallback at index `eventId.arity()`) as a side effect. This happens when every real condition has been resolved or the aggregate YES reaches `RESULT_DENOMINATOR`:

```
ConditionId fallbackConditionId  
    = eventId_.computeConditionId(conditionCount_);  
if (  
    (conditionsResolved_ == conditionCount_ || resultsSum_ ==  
    RESULT_DENOMINATOR)  
    && result[fallbackConditionId].length == 0  
) {  
    // [...]  
    _storeResult(fallbackConditionId, fallbackResult);  
    // [...]  
}
```

The auto-resolution path calls `_storeResult` on `fallbackConditionId` without checking `resolutionPausedAt[fallbackConditionId]`. The pause check that ran in the modifier applied to the real condition being reported, not to the fallback. If an admin has paused resolution specifically for the fallback condition ID, a resolver reporting the final triggering real condition will still cause the fallback to be resolved, overriding the admin's intent.

The same gap exists in the migration path: `_finalizeMigrationResolution` delegates to `_finalizeNegriskResolution`, so a bridge-reported migration condition that triggers the auto-derivation also bypasses the pause.

Impact

An admin who pauses the synthetic Other condition ID to prevent its resolution cannot rely on that pause when a resolver reports the last real condition. The fallback is written unconditionally, and positions in the synthetic Other condition receive a result the admin meant to block.

Recommendations

We recommend checking `resolutionPausedAt[fallbackConditionId]` before writing the auto-derived result in `_finalizeNegriskResolution`:

```
ConditionId fallbackConditionId
    = eventId_.computeConditionId(conditionCount_);
if (
    (conditionsResolved_ == conditionCount_ || resultsSum_ ==
     RESULT_DENOMINATOR)
    && result[fallbackConditionId].length == 0
) {
    require(resolutionPausedAt[fallbackConditionId] == 0, ResolutionIsPaused()
    );

    uint256[] memory fallbackResult = new uint256[](2);
    // [...]
    _storeResult(fallbackConditionId, fallbackResult);
}
```

Alternatively, if auto-resolution of the fallback is intended to be unconditional regardless of the fallback condition's pause state, that behavior should be clearly documented in the NatSpec for `_finalizeNegriskResolution` and in the operator documentation for `pauseResolution`.

Remediation

This issue has been acknowledged by Polymarket. Polymarket has stated that they will only pause real conditions.

- 3.8. An empty `_positionIds` array bypasses the `onlyModuleByPositionIds` authorization check, allowing any caller to emit spurious `TransferBatch` events via `batchMint` and `batchBurn`

Target	PositionManager		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

The `onlyModuleByPositionIds` modifier authorizes callers by iterating over the supplied position IDs and verifying each one is owned by the caller's module or that the caller holds `crossModuleAuth`:

```

modifier onlyModuleByPositionIds(PositionId[] calldata _positionIds) {
    uint256 positionIdsLength = _positionIds.length;
    for (uint256 i = 0; i < positionIdsLength; ++i) {
        uint256 moduleId = _positionIds[i].moduleId();
        if (moduleById[moduleId] != msg.sender) {
            require(crossModuleAuth[msg.sender], Unauthorized());
            break;
        }
    }
    _;
}

```

When `_positionIds` is empty, `positionIdsLength` is zero and the loop body never executes.

Both `batchMint` and `batchBurn` are guarded solely by this modifier. With an empty array, an arbitrary caller reaches the function body unchecked. The balance update loop inside `batchMint` and the `_batchBurn` call inside `batchBurn` iterate over the same empty array, so no ERC1155 storage slots are written and no token balances change. However, `batchMint` unconditionally emits a `TransferBatch` event after the balance loop:

```

// Emit a {TransferBatch} event.
log4(m, add(add(n, n), 0x80),
    0x4a39dc06d4c0dbc64b70af90fd698a233a518aa5d07e595d983b8c0526c8f7fb,
    caller(), 0, _to)

```

With `n = 0`, the emitted event carries empty `ids` and `amounts` arrays. Solady's `_batchBurn`, called by `batchBurn`, similarly emits a `TransferBatch` event when passed empty arrays. As a result, any

externally owned account can call `batchMint(recipient, [], [])` or `batchBurn([], [])` and cause the `PositionManager` to emit `TransferBatch(caller, address(0), recipient, [], [])` or `TransferBatch(caller, caller, address(0), [], [])` respectively, without holding any registered module role or `crossModuleAuth`.

Impact

No on-chain token balances are affected; the calls produce no state change beyond the emitted log. The risk is to off-chain infrastructure. The low likelihood reflects that there is no direct financial incentive to trigger these events; the low impact reflects that the arrays carry no position data and on-chain state is unchanged.

Recommendations

We recommend adding an explicit non-empty check to `onlyModuleByPositionIds`:

```
modifier onlyModuleByPositionIds(PositionId[] calldata _positionIds) {
    uint256 positionIdsLength = _positionIds.length;
    require(positionIdsLength != 0, InvalidArrayLength());
    for (uint256 i = 0; i < positionIdsLength; ++i) {
        // [...]
    }
    _;
}
```

Alternatively, the check can be placed at the top of `batchMint` and `batchBurn` individually. Either placement prevents the empty-array path from reaching the event emission and ensures the modifier always inspects at least one position ID before authorizing the caller.

Remediation

This issue has been acknowledged by Polymarket. Polymarket added the following comment:

This is fine. Offchain indexers will be tracking amount as well.

3.9. Modules with crossModuleAuth can split and merge positions belonging to other modules

Target	PositionManager, BaseModule		
Category	Code Maturity	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The onlyModuleByPositionId modifier in PositionManager provides two authorization paths: a module ID match, or a global crossModuleAuth flag.

```

modifier onlyModuleByPositionId(PositionId _positionId) {
    uint256 moduleId = _positionId.moduleId();
    require(moduleById[moduleId] == msg.sender || crossModuleAuth[msg.sender],
        Unauthorized());
    _;
}

```

When crossModuleAuth is set for a module, the position's embedded module ID is never checked. The module is authorized to mint and burn any position, regardless of which module that position belongs to.

The split and merge functions in BaseModule, which all concrete modules inherit, perform no corresponding check that the supplied _conditionId encodes their own module ID before calling into the PositionManager:

```

function split(address[] calldata _to, ConditionId _conditionId,
    uint256 _amount) external virtual {
    require(_to.length == 2, InvalidArrayLength());

    PositionId positionId0 = _conditionId.computePositionId(0);
    PositionId positionId1 = _conditionId.computePositionId(1);

    POSITION_MANAGER.mint(_to[0], positionId0, _amount);
    POSITION_MANAGER.mint(_to[1], positionId1, _amount);
    // [...]
}

```

A module holding crossModuleAuth can therefore call split or merge with a _conditionId that encodes a different module's ID and the PositionManager will authorize the mint or burn.

The concern is forward-looking. If a future module introduces conditions with a different number of outcomes — three outcomes for a ternary market, for instance — the assumption that `split` always produces two positions (outcomeIndex 0 and 1) becomes module-specific rather than universal. A cross-module authorized module calling `split` on a ternary condition would mint positions for only two of those outcomes, leaving the third unminted, and the collateral accounting invariant of the future module would be violated. The future module would have no way to prevent this, because `split` carries no access control.

Impact

There is no current impact. The existing modules share a binary outcome structure, so cross-module splits produce consistent results regardless of which module holds `crossModuleAuth`. The risk is latent: any future module that cannot rely on every `split` producing exactly its expected outcome set must perform its own validation, and nothing in the current codebase documents or enforces that expectation.

Recommendations

We recommend that each module validate the module ID embedded in `_conditionId` at the entry points of `split`, `merge`, and `redeem`, so that a cross-module authorized module cannot operate on another module's conditions:

```
function split(address[] calldata _to, ConditionId _conditionId,
    uint256 _amount) external virtual {
    require(_conditionId.moduleId() == this.moduleId(), InvalidConditionId());
    // [...]
}
```

Remediation

This issue has been acknowledged by Polymarket, and a fix was implemented in commit [a49a1ce5](#).

3.10. PositionIdLib.encode implicitly assumes ResolutionChain.POLYGON and is named inconsistently with its counterparts

Target	Ids		
Category	Code Maturity	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

EventIdLib and ConditionIdLib each offer two layers of encoding: an inner overload that takes an explicit ResolutionChain argument, and an outer convenience overload that defaults to ResolutionChain.POLYGON by name, making the assumption visible at the call site:

```
// EventIdLib
function encode(uint256 _moduleId, bytes32 _baseHash, uint256 _arity)
    internal pure returns (EventId) {
    return encode(_moduleId, _baseHash, _arity, ResolutionChain.POLYGON);
}

// ConditionIdLib
function encode(uint256 _moduleId, bytes32 _baseHash, uint256 _arity,
    uint256 _conditionIndex) internal pure returns (ConditionId) {
    return encode(_moduleId, _baseHash, _arity, _conditionIndex,
        ResolutionChain.POLYGON);
}
```

PositionIdLib.encode does not follow this pattern. It encodes the position ID in a single overload that omits the RESOLUTION_CHAIN_SHIFT term entirely:

```
function encode(
    uint256 _moduleId,
    uint256 _arity,
    uint256 _conditionIndex,
    uint256 _outcomeIndex,
    bytes memory _data
) internal pure returns (PositionId) {
    bytes32 baseHash = computeBaseHash(_moduleId, _data);

    return PositionId.wrap(
        (_moduleId << MODULE_SHIFT)
        | ((uint256(baseHash) & BASE_HASH_MASK) << BASE_HASH_SHIFT)
```

```
        | ((_arity & ARITY_MASK) << ARITY_SHIFT)
        | ((_conditionIndex & CONDITION_INDEX_MASK) <<
CONDITION_INDEX_SHIFT)
        | (_outcomeIndex & OUTCOME_MASK)
    );
}
```

Because `ResolutionChain.POLYGON` is the only enum member and its underlying value is zero, the omission is currently correct: the resolution chain bits are zero whether the term is present or absent. The problem is one of omission rather than error — nothing in the function body documents that the zero is intentional and corresponds to `POLYGON`, so a developer adding a second `ResolutionChain` value in the future may update `EventIdLib` and `ConditionIdLib` while overlooking `PositionIdLib`, silently producing position IDs whose embedded chain does not match the conditions and events they derive from.

A secondary inconsistency is the function name. Both `EventIdLib` and `ConditionIdLib` expose a data-hashing convenience entry point under the name `encodeFromData`; `PositionIdLib` provides the same operation but names it `encode`.

Impact

There is no current impact. The only defined `ResolutionChain` value is `POLYGON`, which is zero, and `PositionIdLib.encode` produces the same bit pattern that an explicit `ResolutionChain.POLYGON` term would produce. The risk is latent: if a second chain is added, the missing term becomes a silent encoding mismatch that could cause position IDs to be derived on a different chain than the conditions and events they are associated with.

Recommendations

We recommend making the resolution chain explicit in `PositionIdLib.encode` so that the assumption matches the pattern established by `EventIdLib` and `ConditionIdLib`:

```
return PositionId.wrap(
    (_moduleId << MODULE_SHIFT)
    | ((uint256(baseHash) & BASE_HASH_MASK) << BASE_HASH_SHIFT)
    | ((_arity & ARITY_MASK) << ARITY_SHIFT)
    | (uint256(ResolutionChain.POLYGON) << RESOLUTION_CHAIN_SHIFT)
    | ((_conditionIndex & CONDITION_INDEX_MASK) << CONDITION_INDEX_SHIFT)
    | (_outcomeIndex & OUTCOME_MASK)
);
```

We also recommend renaming `PositionIdLib.encode` to `encodeFromData` to match the naming convention used by `EventIdLib.encodeFromData` and `ConditionIdLib.encodeFromData`, all of which hash raw bytes data into the base hash rather than accepting a pre-computed hash.

Remediation

This issue has been acknowledged by Polymarket, and a fix was implemented in commit [449e7bdd](#).

- 3.11. `_processMessage` silently corrupts the caller's `_message` reference via an unaligned in-place slice; `_trimArray` similarly invalidates its caller's array through aliasing

Target	BridgeBase, CombinatorialModule		
Category	Code Maturity	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

BridgeBase._processMessage

To strip the one-byte `MessageType` prefix, `_processMessage` advances the local `_message` pointer by one byte and writes the decremented length at the new offset:

```
assembly ("memory-safe") {
    // payload = _message[1:], update _message in place
    let payloadLength := sub(mload(_message), 1)
    _message := add(_message, 1)
    mstore(_message, payloadLength)
}
```

In Solidity's memory layout, a bytes array stores its length at address `_message` (aligned to a 32-byte word) followed by data at `_message + 32`. After `_message := add(_message, 1)`, the local pointer now refers to `original + 1`. The subsequent `mstore(_message, payloadLength)` writes 32 bytes starting at `original + 1`, which overlaps the last 31 bytes of the original length word and the first byte of the data region. This correctly positions the new length so that the shifted `_message` value refers to a valid-looking payload, but it overwrites the tail of the original length word.

The handlers receive the correctly shifted pointer and operate on a valid payload. The corruption affects only the original reference to `_message.data` held by the caller. In `CcipBridge._ccipReceive`, `_message.data` is passed to `_processMessage` and never read again, so there is no observable impact:

```
function _ccipReceive(Client.Any2EVMMessage memory _message)
    internal override {
        address sender = abi.decode(_message.sender, (address));
        require(peers[_message.sourceChainSelector] == sender,
            UnauthorizedSender());
    }
```

```
_processMessage(_message.messageId, _message.data); // _message.data is
not read after this point
}
```

CombinatorialModule._trimArray

_trimArray truncates an array in place by overwriting its length word with the requested shorter length and returning the same pointer:

```
function _trimArray(PositionId[] memory _arr, uint256 _length)
    internal pure returns (PositionId[] memory) {
    require(_length <= _arr.length, InvalidArrayLength());
    assembly ("memory-safe") {
        mstore(_arr, _length)
    }
    return _arr;
}
```

Because return _arr returns the same memory address, trimmed and the caller's remaining are aliases after the call. The mutation silently truncates the caller's view of remaining to remainingCount entries:

```
PositionId[] memory trimmed = _trimArray(remaining, remainingCount);
newPositionId = _storeLegsFromMemory(trimmed).computePositionId(0);
// remaining.length is now remainingCount, not its original value
```

At both call sites, remaining is not accessed after _trimArray returns, so the aliased mutation has no functional effect. The aligned mstore used here is the standard Solidity pattern for in-place array truncation, unlike the unaligned write in _processMessage; nevertheless, the aliasing is not documented and could mislead a future maintainer who reads remaining after the call.

Impact

There is no current impact in either case. The corrupted _message.data reference in _ccipReceive is never read after _processMessage returns, and the aliased remaining in CombinatorialModule is not accessed after either _trimArray call. The risk is maintenance: a future caller of _processMessage or _trimArray that reads the original reference after the call would observe unexpected behavior without any visible indication at the call site that the reference has been invalidated.

Recommendations

We recommend adding inline comments at the call sites documenting that the original reference is invalidated after the call:

```
// _message.data is passed by reference; _processMessage advances the pointer
// and writes an unaligned length word, invalidating the caller's copy of
// _message.data.
_processMessage(_message.messageId, _message.data);
```

```
// _trimArray updates _arr's length word in place; after this call,
// remaining.length == remainingCount.
PositionId[] memory trimmed = _trimArray(remaining, remainingCount);
```

Remediation

This issue has been acknowledged by Polymarket, and fixes were implemented in the following commits:

- [41b3d692](#) ↗
- [633c6c15](#) ↗

3.12. Splits that produce zero-value tokens can theoretically be griefed by frontrunners

Target	BaseModule, NegRiskModule, CombinatorialModule		
Category	Business Logic	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The position tokens unconditionally allow permissionless minting of paired positions by burning collateral, without even exposing the preimage of the legs hash (for CombinatorialModule). This is by design, to enable the opportunistic minting of tokens corresponding to yet-undeployed conditions. However, one important quirk of this design is that already-settled conditions can still be split on.

When combined with calls to redeem already-settled conditions, and with a sufficiently large flash loan or borrowed capital, this means that at relatively low cost (gas and loan fees), anyone can mint an unlimited number of settled zero-value position tokens.

Also, during the normal usage of this system, the minting of zero-value tokens may be required. For instance, if some conditions of a NegRiskModule position have been finalized, but not others, a user who calls `convert` will necessarily receive some zero-value tokens.

The interaction between these two effects can cause undesired frontrunning opportunities. If a victim user has a pending call to `convert` that produces a zero-value YES token, and an attacker has a large enough financial incentive to interrupt their action, they could produce and donate to them enough of the same token such that the `convert` will fail due to exceeding the integer maximum balance.

Impact

Unless an unlimited-mint zero-fee (or fixed-fee) flash loan facility exists for the collateral tokens, this attack is not feasible. Reaching the 2^{256} balance cap on the PositionManager's ERC-1155 tokens would require so many split-and-redeem round trips, each moving a real-world amount of capital, that they could not fit within any realistic number of blocks.

Recommendations

We recommend adding a try-catch to calls to the PositionManager `mint` function. In the unlikely case where the call reverts, we can then check if the token is a resolved zero-value token, and we can also do a balance check to see if it reverted because it is trying to set the balance to be more

than 2^{256} . If both are the case, then the transaction can proceed, otherwise revert.

Alternatively, documenting this behavior could be enough. Sophisticated users who are concerned about this type of frontrunning can work with block builders to ensure their transactions are not frontrun in a way that causes them to revert.

Remediation

This issue has been acknowledged by Polymarket.

3.13. Modules unconditionally accept incoming ERC1155 transfers but provide no recovery path for positions with a mismatched module ID

Target	BaseModule		
Category	Code Maturity	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

No module exposes a sweep, rescue, or recover function. If a position token whose embedded module ID does not match the receiving module is transferred to that module's address, no code path exists to extract it.

The merge and redeem entry points in BaseModule both pre-burn positions through PositionManager:

```
POSITION_MANAGER.burn(positionId0, _amount); // merge - burns from msg.sender
    (the module)
POSITION_MANAGER.burn(_positionId, _amount); // redeem - burns from msg.sender
    (the module)
```

PositionManager.burn is guarded by onlyModuleByPositionId, which checks that the position's embedded module ID matches the calling module (or that the caller holds crossModuleAuth). If Module A holds a position whose embedded module ID points to Module B, Module A cannot burn it. Calling merge or redeem on Module A with that position ID would revert at the burn call, and the position would remain in Module A's balance indefinitely.

The scenario that leads to permanent loss is a standalone transfer to the wrong module address — one not paired with any consuming call in the same transaction. In practice, this must be a two-step user action, because the correct usage pattern for merge and redeem already requires atomicity: positions sent to a module in a separate transaction can be consumed by any caller before the intended operation is submitted. A user who follows the safe pattern — delivering and consuming in the same transaction — will have a mismatched-position transaction revert rather than leave tokens stranded, because the burn will fail and unwind the entire call.

Impact

There is no current impact from the contract design itself. A user who sends positions to the wrong module address in a standalone transfer will lose those tokens with no recovery mechanism, but this requires a user error that bypasses the intended atomic delivery pattern. No on-chain invariant is broken and no module state is corrupted.

Recommendations

We recommend adding an admin-gated `rescueERC1155` function that forwards arbitrary token IDs to a designated recipient, providing a recovery path for positions that enter a module's balance by mistake. Restricting the receiver callbacks (`onERC1155Received`, `onERC1155BatchReceived`) to reject mismatched module IDs would not be sufficient on its own, because `PositionManager.unsafeTransferFrom` bypasses those callbacks entirely and could still deposit tokens into a module without triggering a revert.

Alternatively, if recovery is intentionally out of scope, the absence of any rescue mechanism should be clearly documented in the NatSpec for `BaseModule`, so that integrators and users are aware that positions transferred to the wrong module address are permanently lost.

Remediation

This issue has been acknowledged by Polymarket. Polymarket added the following comment:

Not adding recovery function at the moment.

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. `ResolutionStatus.Active` is never written to storage

The `ResolutionStatus` enum defines four states — `None`, `Active`, `ArbitrationRequested`, and `Resolved` — but the `Active` variant is never written to storage. A newly registered request leaves its `ResolutionState.status` at the default zero value, which corresponds to `None`. The `_isActiveStatus` helper treats both `None` and `Active` as equivalent:

```
function _isActiveStatus(ResolutionStatus _status)
    internal pure returns (bool) {
    return _status == ResolutionStatus.None || _status ==
        ResolutionStatus.Active;
}
```

The `getRequestState` view function reconstructs the `Active` label in memory before returning it to callers, so on-chain consumers of that view see the intended state. Off-chain code that reads the `resolutionStates` mapping directly, however, observes `None` for conditions that are conceptually active, which can produce an incorrect picture of the lifecycle.

This is likely an intentional gas optimization: skipping a storage write to `Active` on initialization saves one warm SSTORE. The intent would benefit from being explicit in the source. We suggest either renaming `None` to `Active` and using a separate sentinel (such as a zero `targetContract` check) to represent the unregistered case, or adding a NatSpec comment on `ResolutionStatus.None` stating that it is treated as `Active` and that `Active` is never written to storage, with a corresponding note on `_isActiveStatus`.

4.2. `ManagedOptimisticOracleV2` extends the dispute window past liveness expiry by masking Expired requests as Proposed

In the standard `OptimisticOracleV2`, a proposal that survives its liveness period without a dispute transitions to the `Expired` state. Once expired, the request can no longer be disputed — the only permitted action is settlement, which finalizes the proposed value. This makes the liveness window a hard finality boundary: a proposer whose value reaches `Expired` can be certain no further dispute is possible.

`ManagedOptimisticOracleV2` (MOOV2), which is the oracle backend used by `OptimisticOracleModule`, overrides this behavior through `_getStateForDispute`:

```
function _getStateForDispute(...) internal view override returns (State) {
    State state = _getState(...);

    // As the settlement is permissioned, ignoring the expired state allows
    // extending the disputes till settlement.
    if (state == State.Expired) state = State.Proposed;

    return state;
}
```

disputePrice in the parent OptimisticOracleV2 gates on the state returned by _getStateForDispute. Because MOOV2 maps Expired -> Proposed, any caller who meets the bond requirement can dispute a proposal that has already passed its liveness window, as long as a RESOLVER_ROLE holder has not yet called settle. The effective dispute window runs from the moment a proposal is submitted until the moment a permissioned settler finalizes it; the MOOV2 comment acknowledges this explicitly.

In the context of OptimisticOracleModule, a post-liveness dispute triggers the priceDisputed callback, which calls _reRequest and opens a new OOv2 round. A condition whose proposal survived its liveness period can therefore have its resolution restarted by a dispute filed immediately before settlement, delaying finality by at least another full liveness period. Operators managing conditions through OptimisticOracleModule should be aware that dispute activity can occur after liveness expiry.

4.3. ManagedOptimisticOracleV2 validates liveness against minimumDisputeWindow, which can cause setCustomLiveness to revert if the window is raised

ManagedOptimisticOracleV2 overrides _validateLiveness to enforce a lower bound of minimumDisputeWindow rather than the original minimumLiveness:

```
function _validateLiveness(uint256 liveness) internal view override {
    require(liveness >= minimumDisputeWindow,
        LivenessBelowMinimumDisputeWindow());
    super._validateLiveness(liveness);
}
```

setCustomLiveness calls _validateLiveness internally. Every code path in OptimisticOracleModule that creates or re-creates a request — createRequest, _reRequest, and forceReRequest — calls _setupRequest, which calls setCustomLiveness when a non-zero liveness is provided. If minimumDisputeWindow is later raised above the liveness value passed by those callers, setCustomLiveness reverts, causing createRequest, any dispute-triggered

re-request, and any forced re-request to fail for as long as the condition's configured liveness falls below the new window.

This is not an issue under the current configuration: `minimumDisputeWindow` is set to 3600 seconds (one hour) and the intended liveness for requests is also one hour, so the check passes. The concern is operational: an admin raising `minimumDisputeWindow` without coordinating a corresponding update to the liveness values used by `OptimisticOracleModule` would silently break request creation and re-requesting for any condition whose liveness falls below the new threshold.

4.4. Absence of the `crossModuleAuth` flag does not meaningfully restrict module capabilities

In the `PositionManager` contract, the `crossModuleAuth` flag allows modules to mint and burn tokens owned by other modules. The design intent of this flag is to allow some modules to have more privileges than others: since only the `CombinatorialModule` needs to mint and burn positions owned by other modules, only it should be granted this privilege.

However, this assumes that the restriction limits a module's capabilities in a meaningful way, thereby limiting the impact of a single compromised module. But in the current `PositionManager` design, all modules can mint the collateral token at their discretion anyway. So, the compromise of one module already allows an attacker to mint an unlimited amount of collateral, and therefore create any position tokens they want by using that collateral on the split functions implemented by the other non-compromised modules.

If cross-module capabilities should be controlled, then we recommend adopting a design where each module holds in its own account an amount of collateral tokens that backs its outstanding position tokens. Instead of minting and burning collateral, it can transfer collateral to either the `PositionManager` or a separate contract in order to back minted position tokens. This way, the impact of a single compromised module is limited to the collateral that it holds.

Also, see Discussion [4.6](#) for a proposal where the `CombinatorialModule` does not need `crossModuleAuth` anyway.

Alternatively, if the current permissive module system is not to be changed, then we recommend removing the `crossModuleAuth` flag entirely, in order to not overstate the permission boundary.

4.5. Upgrade checks overstate immutability guarantees

In the `BinaryModule`, `NegRiskModule`, and `CombinatorialModule` contracts, the `_authorizeUpgrade` hook implements restrictions on the new implementation that, according to the comment, enforces immutable config compatibility. For instance, in the `BinaryModule`, we

have:

```

/*-----
                                UUPS AUTHORIZATION
-----*/

/// @dev Restricts upgrades to the owner and enforces immutable config
compatibility.
/// @param newImplementation The proposed implementation contract.
function _authorizeUpgrade(address newImplementation)
    internal view override onlyOwner {
    BinaryModule newImpl = BinaryModule(newImplementation);

    if (
        newImpl.moduleId() != ModuleIds.BINARY ||
        address(newImpl.POSITION_MANAGER()) != address(POSITION_MANAGER)
            || address(newImpl.COLLATERAL_TOKEN()) !=
        address(COLLATERAL_TOKEN)
            || address(newImpl.CONDITIONAL_TOKENS()) !=
        address(CONDITIONAL_TOKENS) || newImpl.USDCE() != USDCE
            || newImpl.RESOLUTION_CHAIN() != RESOLUTION_CHAIN
    ) revert IncompatibleImplementation();
    }

```

Although this is a good sanity check to have, the comment overstates the assurance the function provides. Without an unreasonable amount of dynamic introspection, it is not possible to verify that a future implementation actually declares these variables as immutable at the points these static checks read them. For instance, it is possible that an implementation contract incorrectly both has those variables not be immutable, and was incorrectly initialized with the wrong module's parameters, and, if so, it might return the correct values despite not being correct.

We recommend softening the assertion in the comments of the three `_authorizeUpgrade` functions to highlight that it is only a sanity check, rather than stating that it enforces immutability.

4.6. CombinatorialModule does not need crossModuleAuth flag if it retains child positions to back its positions

As discussed in Discussion 4.4.7, the `crossModuleAuth` flag in the current design does not meaningfully restrict the actual cross-module capacities of a module. If the suggestion to control cross-module capabilities is taken, where modules hold their own collateral tokens to back their own positions, then we further suggest that the `CombinatorialModule` also does not need special privileges. In the same way that it would hold its own collateral tokens, it can also hold its own position tokens on the other modules.

More specifically, each of the advanced position manipulation functions already has a desired invariant where the system itself has no directional exposure to any specific resolution. This would narrow that invariant so that the CombinatorialModule, with the position tokens it holds, continues to have no directional exposure.

5. Threat Model

This provides a full threat model description for various functions. As time permitted, we analyzed each function in the contracts and created a written threat model for some critical functions. A threat model documents a given function's externally controllable inputs and how an attacker could leverage each input to cause harm.

Not all functions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that a function is safe.

5.1. Module: BridgeBase.sol

Function: `bridgeCollateral(uint256 _dstChain, uint256 _amount, bytes32 _recipient, bytes calldata _options)`

Initiates a cross-chain collateral transfer. Collateral must be pre-transferred to this bridge contract before calling.

Execution order:

1. `_bridgeSend` dispatches the cross-chain message carrying `(_recipient, _amount)` as the payload. It triggers `_transportSend`, which is not part of this base bridge implementation but is implemented in the concrete bridge contracts.
2. burns the pre-transferred collateral from the bridge's balance.

Inputs

- `_dstChain`
 - **Control:** Fully caller-controlled.
 - **Constraints:** The transport layer implementation enforces chain validity, for example in `CcipBridge`, `_transportSend` reverts if no peer is configured for `_dstChain`.
 - **Impact:** The destination chain for the cross-chain message.
- `_amount`
 - **Control:** Fully caller-controlled.
 - **Constraints:** The bridge must hold $\geq$ `_amount` of `COLLATERAL_TOKEN`. No minimum or maximum enforced in `BridgeBase`.
 - **Impact:** The exact amount of collateral to be bridged.
- `_recipient`
 - **Control:** Fully caller-controlled.
 - **Constraints:** Must be non-zero. Encoded as `bytes32` for cross-chain compatibility.
 - **Impact:** The address that receives the bridged collateral on the destination chain.

- `_options`
 - **Control:** Fully caller-controlled; arbitrary bytes.
 - **Constraints:** Interpreted and validated by the transport layer. Invalid options may cause `_transportSend` to revert.
 - **Impact:** Configures transport-specific parameters.

Branches and code coverage

Intended branches

- Sends collateral message then burns `_amount` from the bridge
 - ☒ Test coverage
- Emits `CollateralBridged` with sender, recipient, amount
 - ☒ Test coverage

Negative behavior

- Reverts `ZeroRecipient` when `_recipient` is `bytes32(0)`
 - ☐ Negative test
- Reverts `SendIsPaused` when send is paused
 - ☒ Negative test
- Reverts `PeerNotSet` when no peer configured for `_dstChain`
 - ☒ Negative test

Function call analysis

- `BridgeBase.bridgeCollateral` -> `BridgeBase._bridgeSend(_dstChain, payload, _options)`
-> `BridgeBase._transportSend(CcipBridge._transportSend -> IRouterClient.ccipSend)`
 - **What is controllable?** `_dstChain`, the encoded payload (containing `_recipient` and `_amount`), and `_options` are all caller-controlled.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value used. The cross-chain message is dispatched here. `msg.value` is forwarded to cover any native fee. If `msg.value` is insufficient, `ccipSend` reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?** Reentrancy from the router back into the bridge could be exploited if the bridge's state is inconsistent at call time, but even if reentered, a reentrant call attempting to reuse the pre-transferred collateral to double-spend it would cause the original call to revert, because there would be insufficient balance to burn. Since this contract holds no collateral beyond the amount

currently being bridged, the reentrancy is not exploitable. Additionally, the router contract is trusted, so there is no risk of malicious actions.

- BridgeBase.bridgeCollateral -> ICollateralToken.burn(_amount)
 - **What is controllable?** _amount.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value.
 - **What happens if it reverts, reenters or does other unusual control flow?** Reverts if the bridge's collateral balance is below _amount.

Function: bridgePositions(uint256 _dstChain, PositionId[] calldata _positionIds, uint256[] calldata _amounts, bytes32 _recipient, bytes calldata _options)

Initiates a cross-chain position transfer. All positions must belong to the same module and be pre-transferred to this bridge contract.

Execution order:

1. _sendPositionsMessage resolves the module address, validates the route and module, then dispatches the cross-chain message via _bridgeSend -> _transportSend. _transportSend is not part of this base bridge implementation; it is implemented in the concrete bridge contracts.
2. unsafeBatchTransferFrom transfers positions to the module address;
3. burnFromBridge burns positions from the module;

Function is payable to support transports that charge a native fee.

Inputs

- _dstChain
 - **Control:** Fully caller-controlled.
 - **Constraints:** only supported _dstChain.
 - **Impact:** Selects the destination chain for the cross-chain message.
- _positionIds
 - **Control:** Fully caller-controlled.
 - **Constraints:** Must be non-empty and must have the same length as _amounts. All entries must share the same moduleId. Only positions from supported modules can be bridged.
 - **Impact:** Determines which positions are bridged.
- _amounts

- **Control:** Fully caller-controlled.
- **Constraints:** Same length as `_positionIds`. Each amount must be covered by the bridge's pre-transferred balance for the corresponding position id.
- **Impact:** Quantity bridged for each position.
- `_recipient`
 - **Control:** Fully caller-controlled.
 - **Constraints:** Must be non-zero. Encoded as `bytes32` to support non-EVM recipients;
 - **Impact:** The address that receives the bridged positions on the destination chain.
- `_options`
 - **Control:** Fully caller-controlled; arbitrary bytes.
 - **Constraints:** Interpreted by the transport layer (`_transportSend`). Invalid options may cause `_transportSend` to revert. No validation in `BridgeBase`.
 - **Impact:** Configures transport-specific parameters.

Branches and code coverage

Intended branches

- Bridges same-module positions: sends message, transfers to module, burns
 - ☒ Test coverage
- Bridges without any send-side condition preparation
 - ☒ Test coverage
- Emits `PositionsBridged` with sender, recipient, position count
 - ☒ Test coverage

Negative behavior

- Reverts `ArrayLengthMismatch` when `_positionIds.length != _amounts.length`
 - ☒ Negative test
- Reverts `NoPositions` when `_positionIds` is empty
 - ☒ Negative test
- Reverts `ZeroRecipient` when `_recipient` is `bytes32(0)`
 - ☐ Negative test
- Reverts `RouteNotSupported` when module not enabled for `_dstChain`
 - ☒ Negative test
- Reverts `PositionsNotSameModule` when positions span modules

- ☒ Negative test
 - Reverts SendIsPaused when send is paused
- ☒ Negative test
 - Reverts ModuleNotConfigured when first position's module is unresolved
- ☐ Negative test

Function call analysis

- BridgeBase.bridgePositions -> BridgeBase._sendPositionsMessage(_dstChain, _positionIds, _amounts, _recipient, _options)
 - **What is controllable?** All inputs are caller-controlled as described above.
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns moduleAddr — the address registered for _positionIds[0].moduleId() in PositionManager.
 - **What happens if it reverts, reenters or does other unusual control flow?** There are no problems here.
- BridgeBase.bridgePositions -> BridgeBase._sendPositionsMessage -> PositionManager.moduleById(_positionIds[0].moduleId())
 - **What is controllable?** The moduleId bits of _positionIds[0] are caller-controlled. A caller can pass any uint8 value as module id.
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns the registered module address for the given id. If the id maps to an unregistered or malicious module, ModuleNotConfigured reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?** There is no problem here.
- BridgeBase.bridgePositions -> BridgeBase._sendPositionsMessage -> BridgeBase._bridgeSend -> BridgeBase._transportSend (CcipBridge._transportSend -> IRouterClient.ccipSend)
 - **What is controllable?** _dstChain, the encoded payload (containing _recipient and all position ids/amounts), and _options are all caller-influenced. The payload is constructed from caller-provided values.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value used. The cross-chain message is dispatched here. msg.value is forwarded to cover any native fee. If msg.value is insufficient, ccipSend reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?** Reentrancy from the router back into the bridge could be exploited if the bridge's state is inconsistent at call time, but even if reentered, a reentrant call attempting to reuse the pre-transferred positions to double-spend them would cause the original call to revert, because there would be insufficient balance to transfer to the module before burning. Since this contract holds

no positions beyond the ones currently being bridged, the reentrancy is not exploitable. Additionally, the router contract is trusted, so there is no risk of malicious actions.

- BridgeBase.bridgePositions ->
PositionManager.unsafeBatchTransferFrom(address(this), moduleAddr, _positionIds, _amounts)
 - **What is controllable?** _positionIds and _amounts are caller-controlled; moduleAddr is registry-derived.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value.
 - **What happens if it reverts, reenters or does other unusual control flow?** Reverts if the bridge holds insufficient balance for any position.
- BridgeBase.bridgePositions ->
BaseModule(moduleAddr).burnFromBridge(_positionIds, _amounts)
 - **What is controllable?** moduleAddr is registry-derived; _positionIds and _amounts are caller-controlled.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value.
 - **What happens if it reverts, reenters or does other unusual control flow?** External call to a trusted in-system module. A maliciously upgraded or replaced module could reenter via this call. However, a malicious module would introduce more critical issues here than reentrancy, since it could bridge positions without limit.

Function: bridgeResult(uint256 _dstChain, ConditionId _conditionId, bytes calldata _options)

Relays an already-resolved condition result to a destination chain. Permissionless — any caller may relay any result, since the data is read directly from the trusted on-chain module and cannot be forged. No tokens are transferred or burned; the function only moves information about the result.

Execution order:

1. resolve module address from _conditionId.moduleId() via PositionManager registry;
2. validate route is enabled;
3. read getResult(_conditionId) from the module and require length == 2 (i.e., already resolved);
4. _bridgeSend encode the condition id and result array into a payload and dispatch cross-chain.

Inputs

- `_dstChain`
 - **Control:** Fully caller-controlled.
 - **Constraints:** Module should be supported on the destination chain.
 - **Impact:** The destination chain for the cross-chain message.
- `_conditionId`
 - **Control:** Fully caller-controlled.
 - **Constraints:** The condition must have a reported result on that module. The function does not validate that `_conditionId` belongs to any particular market, because it relies entirely on the module's `result` storage.
 - **Impact:** The condition id for which the result is being bridged.
- `_options`
 - **Control:** Fully caller-controlled; arbitrary bytes.
 - **Constraints:** Interpreted and validated by the transport layer. Invalid options may cause `_transportSend` to revert.
 - **Impact:** Configures transport-specific parameters.

Branches and code coverage

Intended branches

- Bridges a reported binary result and emits `ResultBridged`
 - ☒ Test coverage
- Bridges a reported `negRisk` result and emits `ResultBridged`
 - ☒ Test coverage

Negative behavior

- Reverts `ModuleNotConfigured` when module unresolved for condition
 - ☒ Negative test
- Reverts `RouteNotSupported` when module not enabled for `_dstChain`
 - ☒ Negative test
- Reverts `ResultNotReported` when `result` length `!= 2`
 - ☒ Negative test
- Reverts `SendIsPaused` when `send` is paused
 - ☒ Negative test

Function call analysis

- BridgeBase.bridgeResult -> PositionManager.moduleById(_conditionId.moduleId())
 - **What is controllable?** _conditionId.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns the registered module address. Reverts if the id maps to address(0). If the conditionId does not exist but the moduleId is valid, the subsequent getResult call will return a zero result, so it is not necessary to validate the conditionId here.
 - **What happens if it reverts, reenters or does other unusual control flow?**
There is no problem here.
- BridgeBase.bridgeResult -> BaseModule(moduleAddr).getResult(_conditionId)
 - **What is controllable?** _conditionId.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns the stored result array. if length != 2 the function will revert. If length == 2, the result is valid and will be bridged.
 - **What happens if it reverts, reenters or does other unusual control flow?**
There is no problem here.
- BridgeBase.bridgeResult -> BridgeBase._bridgeSend(_dstChain, payload, _options) -> BridgeBase._transportSend(CcipBridge._transportSend -> IRouterClient.ccipSend)
 - **What is controllable?** _dstChain and _options are caller-controlled.
 - **If the return value is controllable, how is it used and how can it go wrong?**
No return value used.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Reentrancy is not exploitable here.

5.2. Module: CcipBridge.sol

Function: ccipReceive(Client.Any2EVMMessage calldata message)

CCIP protocol entry point for inbound messages.

Only immutable CCIP Router contract address can call this function. Additionally, inside _ccipReceive, the decoded message.sender is checked against peers[sourceChainSelector], ensuring only registered peer bridges on the source chain are trusted.

After caller and original sender validation, the received message is processed by _processMessage function and routes to one of three handlers:

- _handlePositionsReceive: mints position tokens
- _handleCollateralReceive: mints collateral
- _handleResultReceive: reports a resolution result to a module.
- Unknown MessageType bytes are silently ignored.

Inputs

- `message`
 - **Control:** Constructed by the CCIP router from the source chain transmission; content originates from the source-chain bridge (peer). Effectively trusted only after both `onlyRouter` and `UnauthorizedSender` checks pass.
 - **Constraints:** Must be delivered by the configured CCIP router (`onlyRouter`). The ABI-decoded sender address must equal `peers[sourceChainSelector].data` must be at least 1 byte (first byte is the `MessageType` enum).
 - **Impact:** Determines what action is taken on this chain.
- `message:messageId`
 - **Control:** Set by the CCIP router; not forgeable by end-users. Passed through to `_processMessage` -> handlers -> emitted events only. Not used for access control or state mutation gating.
 - **Constraints:** No on-chain validation; treated as an opaque identifier.
 - **Impact:** Emitted in `PositionReceived`, `CollateralReceived`, `ResultReceived` for indexing purposes.
- `message:sourceChainSelector`
 - **Control:** Set by the CCIP router based on the actual origin chain.
 - **Constraints:** Used as the key for `peers[sourceChainSelector]` lookup, must be registered in the bridge.
 - **Impact:** Determines which peer address is considered trusted for this inbound message.
- `message:sender`
 - **Control:** ABI-encoded address set by the CCIP router from the source-chain transmitter.
 - **Constraints:** ABI-decoded to address `sender`; compared against `peers[sourceChainSelector]`. If the peer was removed or changed since the message was sent, this check will fail.
 - **Impact:** Provides source-bridge authentication.
- `message:data`
 - **Control:** Payload content is set by the source-chain bridge at send time; originates from `BridgePayloads.positions/collateral/result`. If the peer bridge is compromised, arbitrary data could be delivered.
 - **Constraints:** First byte must be a valid `MessageType` value (0, 1, or 2) to trigger any handler; other values are silently ignored. Remaining bytes are ABI-decoded by the handler.
 - **Impact:** Determines the handler branch and all minting/reporting side-effects. A crafted first byte selects the handler; a crafted payload body determines recipient, position ids, amounts, or result values.

- `message:destTokenAmounts`
 - **Control:** Not used by BridgeBase at all.
 - **Constraints:** No validation performed.
 - **Impact:** None — ignored entirely.

Branches and code coverage

Intended branches

- Valid CCIP message from trusted peer processed end-to-end
 - ☒ Test coverage
- Collateral message (`MessageType.COLLATERAL`) received and minted to recipient
 - ☒ Test coverage
- Position message (`MessageType.POSITIONS`) received and minted per position
 - ☒ Test coverage
- Result message (`MessageType.RESULT`) received and reported to module
 - ☒ Test coverage
- Unknown `MessageType` byte silently ignored (no revert, no state change)
 - ☐ Test coverage

Negative behavior

- Caller is not the configured CCIP router reverts `InvalidRouter(msg.sender)`
 - ☐ Negative test
- Sender is not the registered peer for `sourceChainSelector` reverts `UnauthorizedSender`
 - ☒ Negative test
- Receive is paused reverts `ReceiveIsPaused` (via `_processMessage` when `ReceiveUnpaused`)
 - ☐ Negative test
- Positions payload: module not configured reverts `ModuleNotConfigured`
 - ☐ Negative test
- Positions payload: positions span multiple modules reverts `PositionsNotSameModule`
 - ☐ Negative test

Function call analysis

- CcipBridge.ccipReceive -> CcipBridge._ccipReceive(_message)
 - **What is controllable?** message is delivered by the trusted CCIP router; content originates from the peer bridge. Both sourceChainSelector and sender are set by the router and not directly manipulable by users. data content is set by the source-chain bridge.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value. Decodes message.sender via abi.decode -> address.
 - **What happens if it reverts, reenters or does other unusual control flow?** UnauthorizedSender aborts cleanly before any state changes. If CCIP re-delivers a message after a peer was changed/removed, this revert blocks the delivery — funds would be stranded in-flight until the peer is restored or a recovery mechanism handles the stuck message.
- CcipBridge.ccipReceive -> _ccipReceive -> BridgeBase._processMessage -> BridgeBase._handlePositionsReceive -> BaseModule.mintFromBridge [MessageType.POSITIONS path]
 - **What is controllable?** recipient, positionId, and amount for each position come from the payload crafted by the source-chain bridge. If the peer bridge is compromised, arbitrary positions could be minted to arbitrary recipients.
 - **If the return value is controllable, how is it used and how can it go wrong?** mintFromBridge is onlyBridge-gated — the bridge must hold BRIDGE_ROLE on the module. The module id in each position id is checked for consistency across the batch. A mismatched module id in any payload entry (or unregistered module id) reverts the entire call. If any single mintFromBridge cannot succeed, the entire inbound delivery is bricked and must be re-sent as separate messages.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert from any mintFromBridge call in the loop unwinds all preceding mints atomically. mintFromBridge is callback-free, so no reentrancy from the recipient side. A module-level revert would permanently block delivery of the batched message.
- CcipBridge.ccipReceive -> _ccipReceive -> BridgeBase._processMessage -> BridgeBase._handleCollateralReceive -> CollateralToken.mint [MessageType.COLLATERAL path]
 - **What is controllable?** recipient and amount come from the payload. A compromised peer could mint arbitrary collateral amounts to arbitrary addresses.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value. COLLATERAL_TOKEN.mint requires the bridge to hold the minter role. The recipient is not validated for zero address here — a zero recipient would be caught inside CollateralToken.mint.
 - **What happens if it reverts, reenters or does other unusual control flow?** Current CollateralToken (Solady ERC20) mint is callback-free, so no

reentrancy.

- CcipBridge.ccipReceive -> _ccipReceive -> BridgeBase._processMessage -> BridgeBase._handleResultReceive -> BaseModule.reportResult [MessageType.RESULT path]
 - **What is controllable?** conditionId and resultData (the resolution numerators) come from the payload. A compromised peer could report arbitrary results for any condition id on the target chain.
 - **If the return value is controllable, how is it used and how can it go wrong?** reportResult access control on the module determines whether the bridge is authorized to report (bridge must have oracle/result-reporter role). The condition id's embedded module id resolves which module is called. A maliciously crafted conditionId could target an unintended module and a crafted resultData could report incorrect payouts. But there is no problem here in the current implementation.
 - **What happens if it reverts, reenters or does other unusual control flow?** Revert from reportResult blocks this inbound delivery. reportResult is expected to be non-reentrant.

5.3. Module: ChainlinkReporterModule.sol

Function: initializeReporterModule(EventId eventId, bytes calldata data)

Called by the aggregator from within _registerReporterModules during initializeRequest. Decodes the candle window config from data and stores it in eventConfigs[eventId]. The onlyAggregator modifier restricts the caller to the trusted aggregator address. If eventId has been already initialized, the function will revert because of the initOnce modifier. Only binary events (arity == 0) are accepted.

Inputs

- eventId
 - **Control:** Controlled by the aggregator.
 - **Constraints:** Must encode arity == 0 (binary event). The initOnce modifier ensures that this module has not previously been initialized for this event.
 - **Impact:** Keys both the eventConfigs storage write and the requestInitialized flag (via asCondition).
- data
 - **Control:** Operator-originated: the initData field inside the ModuleConfig struct

passed by the operator to `initializeRequest`. The aggregator passes it through without

inspection.

- **Constraints:** Must ABI-decode as `(bytes32 assetPair, uint256 duration, uint256 startTimestamp)`. `startTimestamp + duration` must be strictly greater than `block.timestamp` at call time.
- **Impact:** Determines the candle window stored in `eventConfigs[eventId]`. An operator can set any `assetPair` (including one with no `feedId` configured, which will cause `report()` to revert later with `FeedIdNotSet`), any `duration`, and any future `startTimestamp`. Operators are trusted admin-configured addresses.

Branches and code coverage

Intended branches

- Decodes config, stores `EventConfig`, and marks the binary request initialized

☒ Test coverage

Negative behavior

- Reverts `NotBinaryEvent` when the event arity is not zero
- Reverts `WindowAlreadyEnded` when `startTimestamp + duration <= now`

☒ Negative test

☒ Negative test

- Reverts when caller is not the aggregator

☐ Negative test

- Reverts when the request scope is already initialized

☐ Negative test

Function: `report(bytes32 _requestId)`

Permissions function. Reads start and end prices from the `DataStore`, computes a binary payout (`RESULT_DENOMINATOR` if `endPrice >= startPrice`, else 0), and forwards a single-element result to the aggregator via `reportResult`. The function has no per-request deduplication of its own — the

aggregator enforces AlreadyVoted on the second call.

Inputs

- `_requestId`
 - **Control:** Fully caller-controlled.
 - **Constraints:** Config for the corresponding eventId must have been initialized.
 - **Impact:** Determines the requestId(eventId) the result will be reported for. A caller cannot influence the stored config or the price data — they can only choose which initialized event to trigger reporting for.

Branches and code coverage

Intended branches

- `endPrice >= startPrice` yields a full payout and forwards YES to the aggregator
 - ☒ Test coverage
- `endPrice < startPrice` yields a zero payout (NO outcome)
 - ☒ Test coverage
- Succeeds before window end when both prices already written
 - ☒ Test coverage

Negative behavior

- Reverts RequestNotInitialized when the event was never initialized
 - ☒ Negative test
- Reverts InvalidRequestId when called with a neg-risk sub-condition id
 - ☒ Negative test
- Reverts FeedIdNotSet when no feed id is configured for the asset pair
 - ☒ Negative test
- Reverts GlobalPaused when the module is paused
 - ☒ Negative test
- Reverts when a required price slot is unwritten in the DataStore
 - ☒ Negative test

Function call analysis

- ChainlinkReporterModule.report -> ChainlinkReporterModule._getEventWindow
 - **What is controllable?** eventId is caller-derived. The function reads eventConfigs[eventId] from storage, which was written by the operator at init time.
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns assetPair, duration, startTimestamp, endTimestamp from trusted storage. The caller cannot influence these values.
 - **What happens if it reverts, reenters or does other unusual control flow?** Reverts if the eventId is not initialized.
- ChainlinkReporterModule.report -> ChainlinkReporterModule._getPrice -> IDataStore.read
 - **What is controllable?** feedId comes from feedIds[assetPair] (admin-set storage); timestamps come from EventConfig (operator-set). priceSource is admin-set. None are caller-controllable. Called twice: once for startTimestamp, once for endTimestamp.
 - **If the return value is controllable, how is it used and how can it go wrong?** The returned price determines the payout (full YES or zero NO).
 - **What happens if it reverts, reenters or does other unusual control flow?** External call to the immutable DATA_STORE. Reverts if the slot has not been written yet. been written yet.
- ChainlinkReporterModule.report -> OracleAggregator.reportResult
 - **What is controllable?** _requestId is the original caller-supplied; result is computed from storage-derived prices. aggregator is admin-set and not caller-controllable.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value. The aggregator casts a vote for this module+result combination. If the aggregator's reporterThreshold is 1, this single call is sufficient to propose

an

outcome and open the dispute window.

- **What happens if it reverts, reenters or does other unusual control flow?**
The aggregator could revert with: NotRegisteredModule, RequestNotActive, AlreadyVoted, InvalidResultLength and InvalidResultSum.

5.4. Module: CombinatorialModule.sol

Function: burnFromBridge(PositionId[] calldata _positionIds, uint256[] calldata _amounts)

Bridge-only batch burn of combinatorial positions held by this module. Positions must be transferred to the module before calling. Validates every id in the loop (`moduleId == COMBINATORIAL, outcomeIndex < 2`), then issues a single `batchBurn` from the module's own balance, and emits `BridgePositionsBurned`. Unlike `mintFromBridge`, there is no per-element zero-amount short-circuit — a zero amount passes validation and is forwarded to `batchBurn` as a no-op burn.

Inputs

- `_positionIds`
 - **Control:** Controlled by the bridge contract.
 - **Constraints:** Each element must have `moduleId() == COMBINATORIAL` and `outcomeIndex() < 2`. An empty array is valid and produces a no-op.
 - **Impact:** Determines which positions are burned from the module's balance.
- `_amounts`
 - **Control:** Controlled by the bridge contract.
 - **Constraints:** Must match `_positionIds` in length (enforced downstream in `PositionManager._batchBurn`). The module must hold at least `_amounts[i]` of each `_positionIds[i]`. Zero amounts are allowed and are no-ops for that element.
 - **Impact:** Quantity burned per position.

Branches and code coverage

Intended branches

- bridge burns single combinatorial position held by module
- ☑ Test coverage
- bridge burns batch of combinatorial positions

☒ Test coverage

Negative behavior

- position not from combinatorial module reverts InvalidConditionModule

☒ Negative test

- non-bridge caller reverts Unauthorized (onlyBridge)

☐ Negative test

Function call analysis

- CombinatorialModule.burnFromBridge -> CombinatorialModule._validateCombinatorialPositionId(_positionIds[i]) [called once per element]
 - **What is controllable?** Each _positionIds[i] is bridge-controlled.
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns nothing; reverts InvalidConditionModule if moduleId() != COMBINATORIAL, or InvalidOutcomeIndex if outcomeIndex() >= 2.
 - **What happens if it reverts, reenters or does other unusual control flow?** There are no problems here.
- CombinatorialModule.burnFromBridge -> PositionManager.batchBurn(_positionIds, _amounts)
 - **What is controllable?** _positionIds and _amounts are bridge-controlled.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value. Burns from this module's balance. batchBurn is gated by onlyModuleByPositionIds, and all ids are COMBINATORIAL positions owned by this module, so the call is authorized and cannot touch other modules' namespaces.
 - **What happens if it reverts, reenters or does other unusual control flow?** Trusted in-system external call; burns are callback-free (no onERC1155Received), so no reentrancy. Reverts if the module's balance is insufficient, or if the arrays differ in length.

Function: `compress(address _to, PositionId _positionId, uint256 _amount)`

Partially redeems a combinatorial position when at least one (but not necessarily all) of its legs has resolved. Requires at least one resolved leg.

Inputs

- `_to`
 - **Control:** Fully caller-controlled; the function is permissionless.
 - **Constraints:** Must be non-zero.
 - **Impact:** Receives `collateralOut` of `collateral` and/or `positionAmount`.
- `_positionId`
 - **Control:** Fully caller-controlled.
 - **Constraints:** `outcomeIndex()` must be 0 or 1. Its condition must be prepared. At least one of its legs must be resolved. The module must hold $\geq$ `_amount` of this position.
 - **Impact:** Determines the full leg set to scan, the YES/NO payout path, and the resulting outputs.
- `_amount`
 - **Control:** Fully caller-controlled.
 - **Constraints:** The module must hold $\geq$ `_amount` of `_positionId`. If `_amount = 0` the call succeeds silently with all output amounts zero.
 - **Impact:** Amount of position to compress.

Branches and code coverage

Intended branches

- YES: resolved true condition removed, reduced YES minted
 - ☒ Test coverage
- YES: resolved false condition makes position worthless
 - ☒ Test coverage
- YES: all legs resolved true -> pure collateral
 - ☒ Test coverage
- YES: fractional resolved leg scales down amount
 - ☒ Test coverage
- YES: fractional after unresolved leg scales down
 - ☒ Test coverage
- NO: resolved false condition mints collateral
 - ☒ Test coverage
- NO: resolved true condition reduces position
 - ☒ Test coverage
- NO: all legs resolved true -> worthless

- ☒ Test coverage
- NO: fractional leg mints collateral and reduced NO
- ☒ Test coverage
- fractional dust does not leak collateral
- ☒ Test coverage

Negative behavior

- no resolved legs reverts PositionNotCompressible
- ☒ Negative test
- outcomeIndex >= 2 reverts InvalidOutcomeIndex
- ☐ Negative test
- legs not prepared reverts ConditionNotPrepared
- ☐ Negative test

Function call analysis

- CombinatorialModule.compress -> CombinatorialModule._getConditionPayout(_leg)
-> PositionManager.moduleById(_leg.moduleId())
 - **What is controllable?** Nothing.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns the underlying module address for the leg. If the leg's module id does not correspond to any registered module, moduleById returns address(0), and the subsequent BaseModule(address(0)).getResult() call would revert. The address of the untrusted module cannot be returned here, because registering the module is a privileged operation.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Trusted in-system view call.
- CombinatorialModule.compress -> CombinatorialModule._getConditionPayout(_leg)
-> BaseModule(module).getResult(_leg.conditionId())
 - **What is controllable?** Nothing is controlled here. The module address is received from the trusted PositionManager registry; the condition id comes from the stored leg.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns the resolution result array. Length != 2 -> treated as unresolved ((false, 0)). A malicious module cannot be called here.
 - **What happens if it reverts, reenters or does other unusual control flow?**
View call on a trusted module; no state changes possible.
- CombinatorialModule.compress -> FixedPointMathLib.mulDiv(payoutFactor, conditionPayout, RESULT_DENOMINATOR) / mulDivUp [called per resolved leg, then

once on `_amount`]

- **What is controllable?** Nothing.
- **If the return value is controllable, how is it used and how can it go wrong?**
YES rounds down, NO rounds up for `positionAmount`, so `collateralOut = _amount - positionAmount` is never negative and never over-pays.
- **What happens if it reverts, reenters or does other unusual control flow?**
There are no problems here.
- `CombinatorialModule.compress` ->
`CombinatorialModule._storeLegsFromMemory(trimmed)` [called at most once, only if reduced position exists]
 - **What is controllable?** Nothing
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns the `ConditionId` for the reduced leg set; used to compute `newPositionId`.
 - **What happens if it reverts, reenters or does other unusual control flow?**
There are no problems here.
- `CombinatorialModule.compress` -> `CollateralToken.mint(_to, collateralOut)`
[conditional: only when `collateralOut > 0`]
 - **What is controllable?** `_to` and `collateralOut`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
No return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
There are no problems here.
- `CombinatorialModule.compress` -> `PositionManager.mint(_to, newPositionId, positionAmount)` [conditional: only when `newPositionId != 0`]
 - **What is controllable?** `_to` and `positionAmount`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
No return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Trusted in-system call. Reverts on zero `_to` or overflow.
- `CombinatorialModule.compress` -> `PositionManager.burn(_positionId, _amount)`
 - **What is controllable?** `_positionId` and `_amount`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
No return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Trusted in-system call; no reentrancy. Reverts if the module's balance of `_positionId` is below `_amount`.

Function: `convertToYesBasket(address[] calldata _to, PositionId _fullNoPositionId, uint256 _amount)`

Decomposes a full NO combinatorial position $NO(c_1^c c_2^c \dots^c c_d)$ into its canonical YES basket of d positions. The full NO must be pre-transferred to the module.

Inputs

- `_to`
 - **Control:** Fully caller-controlled; the function is permissionless.
 - **Constraints:** `_to.length` must equal `fullLegs.length`. Non-zero addresses.
 - **Impact:** `_to[i]` receives `_amount` of basket position `i`.
- `_fullNoPositionId`
 - **Control:** Fully caller-controlled.
 - **Constraints:** Its `outcomeIndex()` must be 1 (NO). Its `conditionId()` must be a prepared combinatorial condition. The module must hold $\geq$ `_amount` of this position.
 - **Impact:** Determines the full leg set from which the basket is derived; the position is burned in exchange for the basket.
- `_amount`
 - **Control:** Fully caller-controlled.
 - **Constraints:** The module must hold $\geq$ `_amount` of `_fullNoPositionId`.
 - **Impact:** Exact quantity minted for every basket YES position (1:1 ratio) and burned from the full NO.

Branches and code coverage

Intended branches

- NO(Q) converted into canonical YES basket positions
 - ☒ Test coverage
- single-condition basket conversion
 - ☒ Test coverage

Negative behavior

- position not NO (`outcomeIndex != 1`) reverts `InvalidOutcomeIndex`
 - ☒ Negative test
- legs not prepared reverts `ConditionNotPrepared`

- ☑ Negative test
- `_to.length != fullLegs.length` reverts `InvalidArrayLength`
- ☑ Negative test

Function call analysis

- `CombinatorialModule.convertToYesBasket` ->
`CombinatorialModule._prepareYesBasketPositionIds(fullLegs)` ->
`CombinatorialModule._storeLegsFromMemory` (called `d` times)
 - **What is controllable?** Nothing
 - **If the return value is controllable, how is it used and how can it go wrong?**
 Returns `basketPositionIds[]`, an array of `d` YES position ids. The ids are used as the mint targets in the subsequent loop.
 - **What happens if it reverts, reenters or does other unusual control flow?**
 There are no problems here.
- `CombinatorialModule.convertToYesBasket` -> `PositionManager.mint(_to[i], basketPositionIds[i], _amount)` [called `d` times]
 - **What is controllable?** `_to[i]` and `_amount`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
 No return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
 Reverts on a zero `_to[i]` or on a balance overflow.
- `CombinatorialModule.convertToYesBasket` -> `PositionManager.burn(_fullNoPositionId, _amount)`
 - **What is controllable?** `_fullNoPositionId` and `_amount`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
 No return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
 Trusted in-system external call; callback-free. Reverts if the module's full-NO balance is below `_amount`.

Function: `extract(address[] calldata _to, PositionId _fullNoPositionId, uint256 _conditionIndex, uint256 _amount)`

Permissionless partial decomposition of a NO combinatorial position: $NO(P^d) \rightarrow NO(P) + YES(P^{not(d)})$, where `d` is the leg at `_conditionIndex` in the full leg set, `P` is the remaining legs, and `not(d)` is `d`'s flipped outcome (same market, opposite side). Follows the pre-transfer pattern: the full NO must already be held by the module. Derives and registers both child conditions (idempotent), then mints `_amount` of `NO(P)` to `_to[0]` and `YES(P^{not(d)})` to `_to[1]`, burns `_amount` of the full NO, and emits `Extracted`. The decomposition is value-preserving: $NO(P^d)$ pays 1 if P^d is false (either `P` is false -> `NO(P)` pays; or `P` true but `d` false -> `YES(P^{not(d)})` pays; mutually

exclusive and exhaustive). The inverse operation is inject.

Inputs

- `_to`
 - **Control:** Fully caller-controlled.
 - **Constraints:** Must have length exactly 2, interpreted as `[NO(P) recipient, YES(Pnot(d)) recipient]`. A zero address reverts.
 - **Impact:** `_to[0]` receives `NO(P)`, `_to[1]` receives `YES(Pnot(d))`.
- `_fullNoPositionId`
 - **Control:** Fully caller-controlled.
 - **Constraints:** Its `outcomeIndex()` must be 1(NO). Its `conditionId()` must be a prepared combinatorial condition with at least 2 legs. The module must hold `>= _amount` of it.
 - **Impact:** Supplies the full leg set from storage; the extracted leg `d` and all derived conditions are computed from it.
- `_conditionIndex`
 - **Control:** Fully caller-controlled.
 - **Constraints:** Must be `< fullLegs.length`, else `ConditionIndexOutOfRange`. Selects a leg **by position in the stored sorted array** — the caller does not supply the leg id directly, only its index. The same market cannot appear twice in a canonical set, so each valid index selects a unique market.
 - **Impact:** Determines which leg `d` is removed to form `reduced` and whose flip `not(d)` is inserted to form `residual`. Choosing different indices from the same full condition yields different (but equally valid) decompositions.
- `_amount`
 - **Control:** Fully caller-controlled.
 - **Constraints:** The module must hold `>= _amount` of `_fullNoPositionId` (pre-transferred), otherwise `PositionManager.burn` reverts.
 - **Impact:** Quantity of each output position minted and of the full NO burned (1 full : 1 reduced NO : 1 residual YES).

Branches and code coverage

Intended branches

- `NO(Pd)` extracted into `reduced NO(P) + residual YES(Pnot(d))`
- ☑ Test coverage
- full decomposition by repeated extraction

- ☑ Test coverage
- used in multicall collateral extraction flow
- ☑ Test coverage

Negative behavior

- position not NO (outcomeIndex != 1) reverts InvalidOutcomeIndex
- ☑ Negative test
- _to.length != 2 reverts InvalidArrayLength
- ☑ Negative test
- legs not prepared reverts ConditionNotPrepared
- ☑ Negative test
- single-condition (length < 2) reverts InvalidConditionSet
- ☑ Negative test
- conditionIndex out of range reverts ConditionIndexOutOfRange
- ☑ Negative test

Function call analysis

- CombinatorialModule.extract -> CombinatorialModule._removeLeg(fullLegs, _conditionIndex)
 - **What is controllable?** _conditionIndex is caller-controlled; fullLegs is read from storage and selected by _fullNoPositionId.
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns a new memory array (reduced) with leg d removed, preserving the ordering of the remaining legs. The result is used as the leg set for the reduced condition and as the base for _insertLeg.
 - **What happens if it reverts, reenters or does other unusual control flow?** There are no problems here.
- CombinatorialModule.extract -> CombinatorialModule._insertLeg(reduced, _flipLeg(d))
 - **What is controllable?** d derives from fullLegs[_conditionIndex] and only the _conditionIndex is caller-controlled.
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns residual = reduced with not(d) sorted-inserted. This operation is safe-by-construction: d was just removed from reduced, so d's market is absent; inserting not(d) adds that market back exactly once.
 - **What happens if it reverts, reenters or does other unusual control flow?** There are no problems here.
- CombinatorialModule.extract -> CombinatorialModule._storeLegsFromMemory(reduced) /

`_storeLegsFromMemory(residual)` [called twice]

- **What is controllable?** The leg arrays are fully determined by `fullLegs` and `_conditionIndex`.
- **If the return value is controllable, how is it used and how can it go wrong?** Returns the `conditionId` for each child set (used to derive the output position ids). Both calls are idempotent — if a child condition was already registered by a prior `extract/splitOnCondition`, the existing entry is returned without overwriting. Only length bounds ($1 \leq \text{length} \leq \text{MAX_LEGS}$) are checked.
- **What happens if it reverts, reenters or does other unusual control flow?** No untrusted external call. Reverts if `length == 0` or `length > MAX_LEGS`. In practice these branches are unreachable.
- `CombinatorialModule.extract -> PositionManager.mint(_to[0], reducedNoPositionId, _amount) / PositionManager.mint(_to[1], residualYesPositionId, _amount)`
 - **What is controllable?** `_to[0]/_to[1]` and `_amount` are caller-controlled
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value.
 - **What happens if it reverts, reenters or does other unusual control flow?** Trusted in-system external call; `PositionManager.mint` is callback-free, so `_to` is no reentrancy vector. Reverts only on zero recipient or balance overflow.
- `CombinatorialModule.extract -> PositionManager.burn(_fullNoPositionId, _amount)`
 - **What is controllable?** `_fullNoPositionId` and `_amount` are caller-controlled.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value. Burns from the module's own balance.
 - **What happens if it reverts, reenters or does other unusual control flow?** Trusted in-system external call; callback-free, no reentrancy. Reverts if the module's pre-transferred full-NO balance is below `_amount`.

Function: `inject(address _to, PositionId _fullNoPositionId, uint256 _conditionIndex, uint256 _amount)`

Inverse of `extract`: reconstructs a full NO position from its two components, $\text{NO}(P) + \text{YES}(P^{\text{not}}(d)) \rightarrow \text{NO}(P^d)$, where d is the leg at `_conditionIndex` in the full leg set and `not(d)` is its flipped outcome. Follows the pre-transfer pattern: `NO(P)` (reduced) and `YES(P^not(d))` (residual) must already be held by the module. Mints `_amount` of the target `NO(P^d)` to `_to`, burns both components via `_burnPair`, and emits `Injected`.

Inputs

- `_to`

- **Control:** Fully caller-controlled; the function is permissionless.
 - **Constraints:** A zero address reverts
 - **Impact:** Receives `_amount` of the reconstructed `NO(P^d)` full position.
- `_fullNoPositionId`
 - **Control:** Fully caller-controlled.
 - **Constraints:** Its `outcomeIndex()` must be 1(NO). Its `conditionId()` must be a prepared combinatorial condition with at least 2 legs.
 - **Impact:** Determines the full leg set from which `d` is extracted and the reduced/residual ids recomputed; it is also the position minted to `_to`.
- `_conditionIndex`
 - **Control:** Fully caller-controlled.
 - **Constraints:** Must be `< fullLegs.length`. Selects which leg `d` to remove from the full set to derive reduced and residual.
 - **Impact:** Determines `d`,

and hence `reducedNoPositionId` and `residualYesPositionId` that must be pre-transferred to the module.
- `_amount`
 - **Control:** Fully caller-controlled.
 - **Constraints:** The module must hold `>= _amount` of both `reducedNoPositionId` and `residualYesPositionId`.
 - **Impact:** Quantity of the full NO minted and of each component burned.

Branches and code coverage

Intended branches

- `reduced NO(P) + residual YES(P^not(d))` injected into `NO(P^d)`

☒ Test coverage

Negative behavior

- `position not NO (outcomeIndex != 1)` reverts `InvalidOutcomeIndex`

☒ Negative test

- `legs not prepared` reverts `ConditionNotPrepared`

☒ Negative test

- `single-condition (length < 2)` reverts `InvalidConditionSet`

☐ Negative test

- `conditionIndex out of range` reverts `ConditionIndexOutOfRange`

☐ Negative test

Function call analysis

- CombinatorialModule.inject -> CombinatorialModule._removeLeg(fullLegs, _conditionIndex)
 - **What is controllable?** _conditionIndex is caller-controlled; fullLegs is read from storage.
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns reduced — the full leg set with leg d at _conditionIndex removed. Used as input to _insertLeg and to getConditionId.
 - **What happens if it reverts, reenters or does other unusual control flow?** There are no problems here.
- CombinatorialModule.inject -> CombinatorialModule._insertLeg(reduced, _flipLeg(d))
 - **What is controllable?** d is fully determined by fullLegs[_conditionIndex]; only the index is caller-controlled.
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns residual — reduced with not(d) sorted-inserted.
 - **What happens if it reverts, reenters or does other unusual control flow?** There are no problems here.
- CombinatorialModule.inject -> CombinatorialModule.getConditionId(reduced) / getConditionId(residual) [called twice]
 - **What is controllable?** The leg arrays are fully determined by fullLegs and _conditionIndex.
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns the conditionId for each child set that is used to compute reducedNoPositionId and residualYesPositionId (the positions to burn).
 - **What happens if it reverts, reenters or does other unusual control flow?** There are no problems here.
- CombinatorialModule.inject -> PositionManager.mint(_to, _fullNoPositionId, _amount)
 - **What is controllable?** _to and _amount are caller-controlled; _fullNoPositionId is caller-controlled.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value.
 - **What happens if it reverts, reenters or does other unusual control flow?** Trusted in-system external call; PositionManager.mint is callback-free, so _to is no reentrancy vector. Reverts only on zero recipient or balance overflow.
- CombinatorialModule.inject -> CombinatorialModule._burnPair(reducedNoPositionId, residualYesPositionId, _amount) -> PositionManager.batchBurn(ids, amounts)
 - **What is controllable?** _amount is caller-controlled; the burned ids are fully determined by fullLegs and _conditionIndex.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value.

- **What happens if it reverts, reenters or does other unusual control flow?**
Trusted in-system external call; burns are callback-free, no reentrancy.
Reverts if the module's balance of either component is below `_amount`.

Function: `merge(address _to, ConditionId _conditionId, uint256 _amount)`

Inverse of `split`, following the pre-transfer pattern: equal amounts of the YES and NO combinatorial positions must already be held by the module before the call. Mints `_amount` of collateral (pUSD) to `_to`, then burns `_amount` of each of the YES/NO positions from the module (via `_burnPair`), and emits `PositionsMerged`.

Inputs

- `_to`
 - **Control:** Fully caller-controlled; the function is permissionless.
 - **Constraints:** A zero address reverts inside `CollateralToken.mint` (Solady `ERC20_mint`).
 - **Impact:** Receives the `_amount` of collateral released by the merge.
- `_conditionId`
 - **Control:** Fully caller-controlled.
 - **Constraints:** `_validateCombinatorialConditionId` requires `moduleId() == COMBINATORIAL` (else `InvalidConditionModule`). Does not need legs registration — only the YES/NO position ids are derived from it.
 - **Impact:** Selects which YES/NO combinatorial position ids are burned.
- `_amount`
 - **Control:** Fully caller-controlled.
 - **Constraints:** The module must hold `>= _amount` of both the YES and NO positions (pre-transferred), otherwise `PositionManager.batchBurn` reverts on insufficient balance.
 - **Impact:** Amount of collateral minted and amount of each position burned (1:1).

Branches and code coverage

Intended branches

- pre-transferred YES+NO combinatorial positions merged into collateral
 - ☒ Test coverage
- split then merge roundtrip restores collateral
 - ☒ Test coverage

Negative behavior

- conditionId not from combinatorial module reverts InvalidConditionModule

☑ Negative test

Function call analysis

- CombinatorialModule.merge -> CombinatorialModule._validateCombinatorialConditionId(_conditionId)
 - **What is controllable?** _conditionId.
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns nothing; only reverts InvalidConditionModule unless the id's module byte is COMBINATORIAL.
 - **What happens if it reverts, reenters or does other unusual control flow?** There are no problems here.
- CombinatorialModule.merge -> ConditionIdLib.computePositionId(_conditionId, 0) / ConditionIdLib.computePositionId(_conditionId, 1)
 - **What is controllable?** _conditionId is controlled; the outcome index is fixed (0 for YES, 1 for NO).
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns the deterministic YES/NO position ids that will be burned. The return value is fully determined by _conditionId; nothing beyond the condition choice can be steered.
 - **What happens if it reverts, reenters or does other unusual control flow?** Pure library, deterministic, no external call, cannot revert or reenter.
- CombinatorialModule.merge -> CollateralToken.mint(_to, _amount)
 - **What is controllable?** Both _to and _amount are caller-controlled.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value. Mints pUSD to _to (the module holds MINTER_ROLE).
 - **What happens if it reverts, reenters or does other unusual control flow?** Trusted in-system external call. The concrete CollateralToken (pUSD) is a Solady ERC20 whose mint does not make a recipient callback, so there is no reentrancy vector here. This is a property of the current implementation of CollateralToken.
- CombinatorialModule.merge -> CombinatorialModule._burnPair(yesPositionId, noPositionId, _amount) -> PositionManager.batchBurn(ids, amounts)
 - **What is controllable?** _amount is caller-controlled; the burned ids derive from the controlled _conditionId.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value. batchBurn is gated by onlyModuleByPositionIds, and both ids are COMBINATORIAL positions owned by this module, so the burn is

authorized and can only touch this module's namespace.

- **What happens if it reverts, reenters or does other unusual control flow?**

Trusted in-system external call; burns are callback-free (no onERC1155Received), so no reentrancy. It reverts if the module's pre-transferred balance of either position is below `_amount`.

Function: `mergeFromYesBasket(address _to, PositionId _fullNoPositionId, uint256 _amount)`

Inverse of `convertToYesBasket`: reconstructs $NO(c_1^{c_2} \dots c_d)$ from its canonical YES basket of `d` positions. All basket positions must be pre-transferred to the module.

Inputs

- `_to`
 - **Control:** Fully caller-controlled; the function is permissionless.
 - **Constraints:** Must be non-zero.
 - **Impact:** `_to` receives `_amount` of the reconstructed full NO position.
- `_fullNoPositionId`
 - **Control:** Fully caller-controlled.
 - **Constraints:** Its `outcomeIndex()` must be 1 (NO). Its `conditionId()` must be a prepared combinatorial condition. The module does not need to hold this position before the call; it is freshly minted.
 - **Impact:** Determines the full leg set from which the basket is derived; the position is minted to `_to`.
- `_amount`
 - **Control:** Fully caller-controlled.
 - **Constraints:** The module must hold $\geq$ `_amount` of every basket position id.
 - **Impact:** Exact quantity minted for the full NO and burned from each basket YES.

Branches and code coverage

Intended branches

- canonical YES basket merged back into `NO(Q)` position

☒ Test coverage

Negative behavior

- position not NO (`outcomeIndex != 1`) reverts `InvalidOutcomeIndex`

- ☑ Negative test
- legs not prepared reverts ConditionNotPrepared
- ☑ Negative test

Function call analysis

- CombinatorialModule.mergeFromYesBasket -> CombinatorialModule._getYesBasketPositionIds(fullLegs)
 - **What is controllable?** Nothing.
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns basketPositionIds[], the ids of the d basket YES positions to burn.
 - **What happens if it reverts, reenters or does other unusual control flow?** Pure function, no external calls, no storage access, no reentrancy, cannot revert.
- CombinatorialModule.mergeFromYesBasket -> PositionManager.mint(_to, _fullNoPositionId, _amount)
 - **What is controllable?** _to, _amount and _fullNoPositionId.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value.
 - **What happens if it reverts, reenters or does other unusual control flow?** Trusted in-system call; callback-free, no reentrancy. Reverts on zero _to or balance overflow.
- CombinatorialModule.mergeFromYesBasket -> CombinatorialModule._burnMany(basketPositionIds, _amount) -> PositionManager.batchBurn(ids, amounts)
 - **What is controllable?** _amount and basketPositionIds.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value.
 - **What happens if it reverts, reenters or does other unusual control flow?** Trusted in-system call; callback-free, no reentrancy. Reverts if the module's balance of any basket position is below _amount.

Function: mergeOnCondition(address _to, PositionId _parentYesPositionId, ConditionId _conditionId, uint256 _amount)

Inverse of splitOnCondition: collapses one condition out of a combinatorial YES position, $YES(P^Y(m)) + YES(P^N(m)) \rightarrow YES(P)$, where P is the parent leg set and m the condition. Follows the pre-transfer pattern: both child YES positions must already be held by the module. It recomputes the two child condition ids from the parent legs plus m's YES/NO legs, mints _amount of the parent YES to _to, burns _amount of each child via _burnPair, and emits MergedOnCondition. Unlike split, it uses the pure _getConditionSplitIds (recompute only) and

registers nothing new — the child conditions must already exist as positions the caller holds. Mint runs before the burn, so the call is atomic.

Inputs

- `_to`
 - **Control:** Fully caller-controlled; the function is permissionless.
 - **Constraints:** A zero address reverts inside `PositionManager.mint` (`TransferToZeroAddress`).
 - **Impact:** Receives the reconstituted parent YES position.
- `_parentYesPositionId`
 - **Control:** Fully caller-controlled.
 - **Constraints:** Its `outcomeIndex()` must be 0 (YES), else `InvalidOutcomeIndex`. Its `conditionId()` must be an already-prepared combinatorial condition (`legs[parentConditionId].length > 0`), else `ConditionNotPrepared` — which also implies it is a COMBINATORIAL-module position, so the `PositionManager.mint` is authorized.
 - **Impact:** Supplies the parent leg set used to recompute the child ids, and is the position minted.
- `_conditionId`
 - **Control:** Fully caller-controlled.
 - **Constraints:** Must reference a BINARY/NEGRISK market. Must not already be one of the parent legs, mirroring the split direction.
 - **Impact:** Determines which two child condition ids are recomputed and burned.
- `_amount`
 - **Control:** Fully caller-controlled.
 - **Constraints:** The module must hold $\geq$ `_amount` of both child YES positions (pre-transferred), otherwise `PositionManager.batchBurn` reverts on insufficient balance.
 - **Impact:** Quantity of parent minted and of each child burned (kept 1 childYes : 1 childNo : 1 parent).

Branches and code coverage

Intended branches

- `childYes + childNo` merged back into parent YES position
- ☒ Test coverage

- used as a leg in multicall collateral extraction flow

☒ Test coverage

Negative behavior

- parent not YES (outcomeIndex != 0) reverts InvalidOutcomeIndex

☒ Negative test

- parent legs not prepared reverts ConditionNotPrepared

☒ Negative test

- condition from unsupported module reverts InvalidConditionModule

☒ Negative test

- condition already present reverts MarketAlreadyPresent

☐ Negative test

Function call analysis

- CombinatorialModule.mergeOnCondition ->
CombinatorialModule._requireConditionNotPresent(parentLegs, _conditionId)
 - **What is controllable?** _conditionId is caller-controlled; parentLegs is read from storage (legs[parentConditionId] is filled through permissionless prepareCondition).
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns nothing
 - **What happens if it reverts, reenters or does other unusual control flow?**
Internal, memory-only, no external call and no reentrancy.
- CombinatorialModule.mergeOnCondition ->
CombinatorialModule._validateConditionModule(_conditionId.computePositionId(0))
 - **What is controllable?** _conditionId is caller-controlled.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns nothing; reverts InvalidConditionModule unless condition's module byte is BINARY or NEGRISK.
 - **What happens if it reverts, reenters or does other unusual control flow?**
There are no problems here.
- CombinatorialModule.mergeOnCondition ->
CombinatorialModule._getConditionSplitIds(parentLegs, _conditionId)
 - **What is controllable?** _conditionId and (indirectly) parentLegs via _parentYesPositionId.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns the two child condition ids, recomputed by canonically inserting condition's YES/NO leg into the parent legs and hashing. Unlike split's

_prepareConditionSplitIds, this variant is pure and does NOT register/store the child conditions.

- **What happens if it reverts, reenters or does other unusual control flow?**
Pure internal, deterministic, no external call, no reentrancy.
- CombinatorialModule.mergeOnCondition -> PositionManager.mint(_to, _parentYesPositionId, _amount)
 - **What is controllable?** _to and _amount are controlled.
 - **If the return value is controllable, how is it used and how can it go wrong?**
No return value. Mint is gated by onlyModuleByPositionId; the parent is a COMBINATORIAL position owned by this module, so the call is authorized and cannot mint another module's positions.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Trusted in-system external call; PositionManager.mint is callback-free, so _to is no reentrancy vector. Reverts only on a zero recipient or balance overflow.
- CombinatorialModule.mergeOnCondition -> CombinatorialModule._burnPair(childYesPositionId, childNoPositionId, _amount) -> PositionManager.batchBurn(ids, amounts)
 - **What is controllable?** _amount is controlled.
 - **If the return value is controllable, how is it used and how can it go wrong?**
No return value. batchBurn is gated by onlyModuleByPositionIds, and both child ids are COMBINATORIAL positions owned by this module, so the burn is authorized.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Trusted in-system external call; burns are callback-free, so no reentrancy. It reverts if the module's pre-transferred balance of either child is below _amount.

Function: mintFromBridge(address _to, PositionId _positionId, uint256 _amount)

Bridge-only mint of a single combinatorial position to a recipient on the destination chain. Called by the bridge contract after a cross-chain position transfer is received. Validates the position id belongs to this module, then mints _amount to _to (skipping both the mint and the event if _amount == 0, to safely handle replay-tolerant bridge delivery). Unlike the collateral-backed split, this mint is not backed by collateral on this chain — the bridge is trusted to have burned the equivalent position on the source chain before sending this message. No legs registration is performed; the position is mintable regardless of whether its leg set is stored, mirroring the split behavior.

Inputs

- _to

- **Control:** Controlled by the bridge contract (derived from the cross-chain message's recipient field).
- **Constraints:** A zero address reverts inside `PositionManager.mint` (`TransferToZeroAddress`). No other on-chain validation of `_to` is performed — the bridge is trusted to deliver to the correct recipient.
- **Impact:** Receives the minted combinatorial position on this chain.
- `_positionId`
 - **Control:** Controlled by the bridge contract (derived from the bridged position's id in the cross-chain message).
 - **Constraints:** `_validateCombinatorialPositionId` requires `moduleId() == COMBINATORIAL` and `outcomeIndex() < 2`. The legs mapping is NOT checked — the position can be minted even if its leg set has never been registered via `prepareCondition` (same as plain split).
 - **Impact:** Determines which combinatorial position is minted.
- `_amount`
 - **Control:** Controlled by the bridge contract (amount burned on the source chain).
 - **Constraints:** Zero is explicitly allowed and treated as a no-op (no mint, no event). Any nonzero value triggers `PositionManager.mint`, which reverts on balance overflow (`AccountBalanceOverflow`).
 - **Impact:** Quantity of the position minted to `_to`.

Branches and code coverage

Intended branches

- `bridge mints combinatorial position (_amount > 0)`
 - ☒ Test coverage
- `zero amount is a no-op`
 - ☒ Test coverage
- `minted position later redeemed`
 - ☒ Test coverage

Negative behavior

- `non-bridge caller reverts Unauthorized (onlyBridge)`
 - ☒ Negative test
- `position not from combinatorial module reverts InvalidConditionModule`
 - ☒ Negative test

Function call analysis

- `CombinatorialModule.mintFromBridge -> CombinatorialModule._validateCombinatorialPositionId(_positionId)`
 - **What is controllable?** `_positionId` is bridge-controlled.
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns nothing; reverts `InvalidConditionModule` if `moduleId() != COMBINATORIAL`, or `InvalidOutcomeIndex` if `outcomeIndex() >= 2`. This prevents the bridge from minting a position that belongs to another module's namespace.
 - **What happens if it reverts, reenters or does other unusual control flow?** Pure internal, no external call, no reentrancy; revert aborts before the amount check and mint.
- `CombinatorialModule.mintFromBridge -> PositionManager.mint(_to, _positionId, _amount)`
 - **What is controllable?** `_to` and `_amount` are bridge-controlled; `_positionId` is bridge-controlled and has passed the validator.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value. Mint is gated by `onlyModuleByPositionId` in `PositionManager`; since the id is `COMBINATORIAL` and this module registered itself for that namespace, the call is authorized.
 - **What happens if it reverts, reenters or does other unusual control flow?** `PositionManager.mint` is callback-free, so `_to` provides no reentrancy vector. Reverts only on a zero recipient or balance overflow.

Function: `multicall(bytes[] calldata _calls)`

Permissionless function which iterates over `_calls` and delegates each into `address(this)`, accumulating return data. Because the target is hardcoded to `address(this)`, only `CombinatorialModule`'s own external functions can be invoked.

Inputs

- `_calls`
 - **Control:** Fully caller-controlled
 - **Constraints:** Each element must be valid ABI-encoded calldata for an external function on this contract. Privileged subcalls (`addBridge`, `removeBridge`, `mintFromBridge`, `burnFromBridge`, `_authorizeUpgrade`) require the caller to hold the corresponding role, enforced via the preserved `msg.sender`.
 - **Impact:** Determines the complete sequence of operations executed atomically.

Branches and code coverage

Intended branches

- executes multiple self-delegatecalls (two splits)
 - ☒ Test coverage
- chained extract/mergeOnCondition/merge returns collateral
 - ☒ Test coverage

Negative behavior

- inner call failure bubbles up the original revert reason
 - ☒ Negative test

Function call analysis

- CombinatorialModule.multicall -> address(this).delegatecall(_calls[i])
 - **What is controllable?** The entire calldata for each subcall
 - **If the return value is controllable, how is it used and how can it go wrong?** The success bool is used to branch on failure (revert propagation); result bytes are stored in the output array and returned to the caller but not used internally. No subsequent subcall reads a previous subcall's return value — subcalls only communicate via shared module storage and pre-transferred token balances.
 - **What happens if it reverts, reenters or does other unusual control flow?** Reentrancy: delegatecall runs code in this contract's storage context; if any subcall (e.g. a mint path) triggers an ERC1155 receiver hook on a caller-controlled _to, that hook could re-enter multicall on the *real* contract address. However, PositionManager.mint is callback-free (no onERC1155Received), so the current mint/burn paths are not reentrancy vectors. The pre-transfer pattern also limits what an attacker can set up within a multicall-triggered re-entry.

Function: prepareCondition(PositionId[] calldata _legs)

Permissionlessly registers a combinatorial condition from a canonical leg set, calculates its deterministic ConditionId, and stores the legs (idempotently, emitting once). Moves no tokens and has no access control or resolution effect, since it only seeds the legs mapping which are used by split/merge/redeem functions.

Inputs

- _legs

- **Control:** Fully caller-controlled calldata array; the function is permissionless.
- **Constraints:** `_validateCanonical` requires `1 <= length <= MAX_LEGS (50)`; every leg's `moduleId()` must be `BINARY (1)` or `NEGRISK (2)`; every leg's `outcomeIndex()` must be `< 2`; entries must be strictly ascending by raw `PositionId` (rejects unsorted input and duplicates); and no two legs may share the same `conditionId()` (rejects combining two outcomes of one market). Underlying leg positions are NOT checked for existence — any structurally valid id passes (existence is enforced later at split/mint time).
- **Impact:** Calculates the `conditionId` for the given leg set. Distinct sets cannot alias to one id and a single set cannot be registered under two ids.

Branches and code coverage

Intended branches

- stores canonical legs and emits `CombinatorialConditionPrepared`
☒ Test coverage
- accepts the maximum number of legs (`MAX_LEGS`)
☒ Test coverage
- idempotent: second prepare of same set does not re-emit/overwrite
☐ Test coverage

Negative behavior

- empty leg array reverts `InvalidConditionSet`
☒ Negative test
- more than `MAX_LEGS` reverts `InvalidConditionSet`
☒ Negative test
- non-ascending order reverts `NonCanonicalInput`
☒ Negative test
- conflicting conditions revert `ConflictingConditions`
☒ Negative test
- duplicate condition reverts
☒ Negative test
- leg from unsupported module reverts `InvalidConditionModule`
☒ Negative test
- invalid outcome index reverts `InvalidOutcomeIndex`
☒ Negative test

Function call analysis

- CombinatorialModule.prepareCondition ->
CombinatorialModule._validateCanonical(_legs) ->
CombinatorialModule._validateConditionModule
 - **What is controllable?** The entire _legs array
 - **If the return value is controllable, how is it used and how can it go wrong?**
The function returns nothing.
 - **What happens if it reverts, reenters or does other unusual control flow?**
There are no problems here since the function is internal and pure.
- CombinatorialModule.prepareCondition ->
ConditionIdLib.encodeFromData(ModuleIds.COMBINATORIAL, 0, abi.encode(_legs))
 - **What is controllable?** _legs.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns the deterministic ConditionId used as the legs storage key and later re-validated by _validateCombinatorialConditionId (moduleId must equal COMBINATORIAL). The caller influences it only through the canonical leg set, so distinct logical sets map to distinct ids and any collision reduces to a keccak256 collision (infeasible).
 - **What happens if it reverts, reenters or does other unusual control flow?** It is deterministic, cannot reenter, and does not revert for well-formed memory input.

Function: redeem(address _to, PositionId _positionId, uint256 _amount)

Final settlement of a fully-resolved combinatorial position for collateral.

Inputs

- _to
 - **Control:** Fully caller-controlled; the function is permissionless.
 - **Constraints:** Must be non-zero.
 - **Impact:** _to receives the collateral payout.
- _positionId
 - **Control:** Fully caller-controlled.
 - **Constraints:** Must encode a valid COMBINATORIAL position. Every leg must be resolved. The module must hold $\geq$ _amount of this position.
 - **Impact:** Determines the leg set, YES/NO path, and all payout arithmetic.
- _amount
 - **Control:** Fully caller-controlled.

- **Constraints:** Module must hold $\geq$ `_amount` of `_positionId`. If `_amount = 0`, the call succeeds silently with zero payout.
- **Impact:** Amount of position to redeem.

Branches and code coverage

Intended branches

- YES all legs true -> full payout
 - ☒ Test coverage
- YES any false -> zero payout
 - ☒ Test coverage
- YES later false before all resolved -> terminal zero
 - ☒ Test coverage
- NO any false -> full payout
 - ☒ Test coverage
- NO later false before all resolved -> terminal full
 - ☒ Test coverage
- NO all true -> zero payout
 - ☒ Test coverage
- YES fractional product payout
 - ☒ Test coverage
- NO fractional complement payout
 - ☒ Test coverage
- bridged position redeemed after prepare+resolve
 - ☒ Test coverage

Negative behavior

- not fully resolved reverts `PositionNotRedeemable`
 - ☒ Negative test
- legs not prepared reverts `ConditionNotPrepared`
 - ☒ Negative test

Function call analysis

- CombinatorialModule.redeem ->
 - CombinatorialModule._getPositionPayout(_positionId, _amount) ->
 - CombinatorialModule._getConditionPayout(_leg) ->
 - PositionManager.moduleById(_leg.moduleId())
 - **What is controllable?** Nothing.
 - **If the return value is controllable, how is it used and how can it go wrong?**
 - Returns the underlying module address. An unregistered module id causes moduleById to return address(0), making the subsequent getResult call revert.
 - **What happens if it reverts, reenters or does other unusual control flow?**
 - View call on a trusted PositionManager.
- CombinatorialModule.redeem -> CombinatorialModule._getPositionPayout ->
 - CombinatorialModule._getConditionPayout(_leg) ->
 - BaseModule(module).getResult(_leg.conditionId())
 - **What is controllable?** Nothing.
 - **If the return value is controllable, how is it used and how can it go wrong?**
 - Returns the resolution result. Length != 2 reverts PositionNotRedeemable.
 - **What happens if it reverts, reenters or does other unusual control flow?**
 - View call on a trusted module.
- CombinatorialModule.redeem -> CollateralToken.mint(_to, payout) [conditional: only when payout > 0]
 - **What is controllable?** _to and payout.
 - **If the return value is controllable, how is it used and how can it go wrong?**
 - No return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
 - Trusted in-system call. Reverts on zero _to or overflow.
- CombinatorialModule.redeem -> PositionManager.burn(_positionId, _amount)
 - **What is controllable?** _positionId and _amount.
 - **If the return value is controllable, how is it used and how can it go wrong?**
 - No return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
 - Trusted in-system call; no reentrancy. Reverts if the module's balance of _positionId is below _amount.

Function: split(address[] calldata _to, ConditionId _conditionId, uint256 _amount)

Permissionless split following the pre-transfer pattern: collateral (pUSD) must already be sitting in the module before the call. Mints _amount of the YES and NO combinatorial positions to _to[0] and _to[1], then burns _amount of the module's own collateral (1:1), and emits PositionsSplit.

Mint happens before burn, so the whole call is atomic — an insufficient pre-transferred balance reverts the burn and unwinds the mints.

Inputs

- `_to`
 - **Control:** Fully caller-controlled; the function is permissionless.
 - **Constraints:** Must have length exactly 2 (`InvalidArrayLength` otherwise); interpreted as `[YES recipient, NO recipient]`. A zero address reverts inside `PositionManager.mint(TransferToZeroAddress)`.
 - **Impact:** `_to[0]/_to[1]` receive the minted YES/NO combinatorial positions.
- `_conditionId`
 - **Control:** Fully caller-controlled.
 - **Constraints:** `_validateCombinatorialConditionId` requires `moduleId() == COMBINATORIAL` (else `InvalidConditionModule`). It need not be pre-registered via `prepareCondition` — `split` only derives position ids from it and does not read the legs mapping (that is consumed at redeem time).
 - **Impact:** Selects which YES/NO combinatorial position ids are minted.
- `_amount`
 - **Control:** Fully caller-controlled.
 - **Constraints:** Must be `<=` the module's pre-transferred collateral balance, otherwise `CollateralToken.burn` reverts on insufficient balance.
 - **Impact:** Quantity of each position minted and amount of collateral burned.

Branches and code coverage

Intended branches

- pre-transferred collateral split into YES+NO combinatorial positions, collateral burned
 - ☒ Test coverage
- single-condition combinatorial split
 - ☒ Test coverage
- split then merge roundtrip restores collateral
 - ☒ Test coverage

Negative behavior

- `_to.length != 2` reverts `InvalidArrayLength`
 - ☒ Negative test

- conditionId not from combinatorial module reverts InvalidConditionModule
- ☑ Negative test

Function call analysis

- CombinatorialModule.split -> CombinatorialModule._validateCombinatorialConditionId(_conditionId)
 - **What is controllable?** _conditionId.
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns nothing; it only reverts InvalidConditionModule unless the id's module byte is COMBINATORIAL. This prevents minting positions in another module's namespace.
 - **What happens if it reverts, reenters or does other unusual control flow?** There are no problems here.
- CombinatorialModule.split -> ConditionIdLib.computePositionId(_conditionId, 0) / ConditionIdLib.computePositionId(_conditionId, 1)
 - **What is controllable?** _conditionId is controlled; the outcome index is fixed (0 for YES, 1 for NO).
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns the deterministic YES/NO position ids used as mint targets.
 - **What happens if it reverts, reenters or does other unusual control flow?** Pure library, deterministic, no external call, cannot revert or reenter.
- CombinatorialModule.split -> PositionManager.mint(_to[0], yesPositionId, _amount) / PositionManager.mint(_to[1], noPositionId, _amount)
 - **What is controllable?** The recipient (_to[i]) and _amount are caller-controlled; the position id is derived from the controlled _conditionId.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value. Mint is gated by onlyModuleByPositionId, and CombinatorialModule is the owning module for COMBINATORIAL position ids, so the call is authorized; a caller cannot mint another module's positions here.
 - **What happens if it reverts, reenters or does other unusual control flow?** Trusted in-system external call. PositionManager.mint is callback-free (hand-written assembly, no onERC1155Received), so there is no reentrancy vector via _to. It reverts only on a zero recipient or balance overflow (AccountBalanceOverflow).
- CombinatorialModule.split -> CollateralToken.burn(_amount)
 - **What is controllable?** _amount.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value. Burns _amount from the module's own pUSD balance (the

module holds MINTER_ROLE), consuming the pre-transferred collateral. The YES/NO supply minted is always matched 1:1 by collateral removed.

- **What happens if it reverts, reenters or does other unusual control flow?**
Trusted in-system external call. Reverts if the module's pre-transferred balance is below `_amount`. No reentrancy.

Function: `splitOnCondition(address[] calldata _to, PositionId _parentYesPositionId, ConditionId _conditionId, uint256 _amount)`

Refines a combinatorial YES position by adding one more condition: $YES(P) \rightarrow YES(P \wedge Y(m)) + YES(P \wedge N(m))$, where P is the parent leg set and m is the new condition. Follows the pre-transfer pattern: the parent YES position must already be held by the module. The function derives the two child conditions (parent legs plus the new condition's YES leg, and parent legs plus its NO leg), registers them, mints `_amount` of each child YES to `_to[0]/_to[1]`, burns `_amount` of the parent YES, and emits `SplitOnCondition`. Mints run before the burn, so the call is atomic.

Inputs

- `_to`
 - **Control:** Fully caller-controlled; the function is permissionless.
 - **Constraints:** Must have length exactly 2 (InvalidArrayLength otherwise), interpreted as [childYes recipient, childNo recipient]. A zero address reverts inside `PositionManager.mint(TransferToZeroAddress)`.
 - **Impact:** `_to[0]/_to[1]` receive the two minted child YES positions.
- `_parentYesPositionId`
 - **Control:** Fully caller-controlled.
 - **Constraints:** Its `outcomeIndex()` must be 0 (YES), else `InvalidOutcomeIndex`. Its `conditionId()` must be an already-prepared combinatorial condition (`legs[parentConditionId].length > 0`), else `ConditionNotPrepared`. The module must hold $\geq$ `_amount` of it.
 - **Impact:** Supplies the parent leg set that the new condition is inserted into, and is the position consumed (burned).
- `_conditionId`
 - **Control:** Fully caller-controlled.
 - **Constraints:** Must reference a BINARY/NEGRISK market. `_conditionId` must not already be one of the parent legs. The resulting child leg set must not exceed `MAX_LEGS` (`InvalidConditionSet`, raised in `_storeLegsFromMemory`).
 - **Impact:** The new condition added to the combination; determines both derived child condition ids.
- `_amount`

- **Control:** Fully caller-controlled.
- **Constraints:** The module must hold $\geq$ `_amount` of the parent YES position, otherwise `PositionManager.burn` reverts.
- **Impact:** Quantity of each child minted and of the parent burned (kept 1 parent : 1 childYes : 1 childNo).

Branches and code coverage

Intended branches

- parent YES split into childYes + childNo on new condition

☒ Test coverage

Negative behavior

- `_to.length != 2` reverts `InvalidArrayLength`

☒ Negative test

- parent not YES (`outcomeIndex != 0`) reverts `InvalidOutcomeIndex`

☒ Negative test

- parent legs not prepared reverts `ConditionNotPrepared`

☒ Negative test

- condition already present reverts `MarketAlreadyPresent`

☒ Negative test

- condition from unsupported module reverts `InvalidConditionModule`

☒ Negative test

- `result` exceeds `MAX_LEGS` reverts `InvalidConditionSet`

☒ Negative test

Function call analysis

- `CombinatorialModule.splitOnCondition -> CombinatorialModule._requireConditionNotPresent(parentLegs, _conditionId)`
 - **What is controllable?** `_conditionId` is caller-controlled; `parentLegs` is read from storage (`legs[parentConditionId]` is filled through `permissionless.prepareCondition`).
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns nothing; it only reverts `MarketAlreadyPresent` if any parent leg already belongs to the new market. This prevents building a combination that references the same underlying market twice.

- **What happens if it reverts, reenters or does other unusual control flow?**
Internal, memory-only, no external call and no reentrancy.
- CombinatorialModule.splitOnCondition ->
CombinatorialModule._validateConditionModule(_conditionId.computePositionId(0))
 - **What is controllable?** _conditionId is caller-controlled.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns nothing; reverts InvalidConditionModule unless the new condition's module byte is BINARY or NEGRISK.
 - **What happens if it reverts, reenters or does other unusual control flow?**
There are no problems here.
- CombinatorialModule.splitOnCondition ->
CombinatorialModule._prepareConditionSplitIds(parentLegs, _conditionId) ->
CombinatorialModule._storeLegsFromMemory(childLegs)
 - **What is controllable?** _conditionId and (indirectly) parentLegs via _parentYesPositionId.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns the two child condition ids, derived by canonically inserting the new YES/NO leg into the sorted parent legs and hashing. The ids are used as the mint targets and are registered in the legs mapping.
 - **What happens if it reverts, reenters or does other unusual control flow?** No reentrancy. It reverts InvalidConditionSet if a child leg set would exceed MAX_LEGS.
- CombinatorialModule.splitOnCondition -> PositionManager.mint(_to[0], childYesPositionId, _amount) / PositionManager.mint(_to[1], childNoPositionId, _amount)
 - **What is controllable?** The recipients (_to[i]) and _amount are controlled; the child position ids derive from the controlled inputs.
 - **If the return value is controllable, how is it used and how can it go wrong?**
No return value. Mint is gated by onlyModuleByPositionId; the child ids are COMBINATORIAL positions owned by this module, so the calls are authorized and cannot mint another module's positions.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Trusted in-system external call; PositionManager.mint is callback-free, so _to is no reentrancy vector. Reverts only on a zero recipient or balance overflow.
- CombinatorialModule.splitOnCondition ->
PositionManager.burn(_parentYesPositionId, _amount)
 - **What is controllable?** _parentYesPositionId and _amount.
 - **If the return value is controllable, how is it used and how can it go wrong?**
No return value. burn is gated by onlyModuleByPositionId and burns from the module itself (msg.sender); since the parent is a prepared combinatorial position it is module-owned, so only this module's namespace is touched.

- **What happens if it reverts, reenters or does other unusual control flow?**
Trusted in-system external call; burns are callback-free, so no reentrancy. It reverts if the module's pre-transferred parent balance is below `_amount`.

Function: `unwrap(address _to, PositionId _positionId, uint256 _amount)`

Unwraps a single-condition combinatorial position back to its underlying binary/negrisk position. The combinatorial must have exactly one stored leg. Requires `crossModuleAuth` on `PositionManager` because the minted underlying position belongs to a different module (binary or negrisk). The input combinatorial position must be pre-transferred to the module.

Inputs

- `_to`
 - **Control:** Fully caller-controlled; the function is permissionless.
 - **Constraints:** Must be non-zero.
 - **Impact:** Receives `_amount` of the underlying position.
- `_positionId`
 - **Control:** Fully caller-controlled.
 - **Constraints:** `outcomeIndex()` must be 0 or 1. Its `conditionId()` must map to exactly one stored leg. The module must hold `>= _amount`.
 - **Impact:** Determines the single stored leg and the YES/NO branch that selects the underlying position.
- `_amount`
 - **Control:** Fully caller-controlled.
 - **Constraints:** Module must hold `>= _amount` of `_positionId`.
 - **Impact:** Amount of position to unwrap.

Branches and code coverage

Intended branches

- YES single-condition unwrapped to underlying position
 - ☒ Test coverage
- NO single-condition unwrapped to flipped underlying position
 - ☒ Test coverage
- YES of N-side condition unwrapped
 - ☒ Test coverage
- wrap then unwrap roundtrip

☒ Test coverage

Negative behavior

- multi-condition position reverts SingleConditionRequired

☒ Negative test

- outcomeIndex >= 2 reverts InvalidOutcomeIndex

☐ Negative test

Function call analysis

- CombinatorialModule.unwrap -> CombinatorialModule._flipLeg(storedLegs[0]) [only for NO path]
 - **What is controllable?** Nothing.
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns the leg with its outcome index flipped.
 - **What happens if it reverts, reenters or does other unusual control flow?** There are no problems here.
- CombinatorialModule.unwrap -> PositionManager.mint(_to, underlyingPositionId, _amount)
 - **What is controllable?** _to and _amount.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value.
 - **What happens if it reverts, reenters or does other unusual control flow?** Trusted in-system call. Reverts on zero _to or overflow.
- CombinatorialModule.unwrap -> PositionManager.burn(_positionId, _amount)
 - **What is controllable?** _positionId, _amount.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value.
 - **What happens if it reverts, reenters or does other unusual control flow?** Trusted in-system call; no reentrancy. Reverts if the module's balance of _positionId is below _amount.

Function: wrap(address _to, PositionId _underlyingPositionId, uint256 _amount)

Wraps an underlying binary/negrisk position into a single-condition combinatorial YES position. Always produces a YES (outcomeIndex=0) regardless of the input outcome: wrapping YES(m) yields combinatorial YES([Y(m)]); wrapping NO(m) yields combinatorial YES([N(m)]). The stored leg is _underlyingPositionId itself. _validateConditionModule enforces that the underlying belongs to the BINARY or NEGRISK module. Requires crossModuleAuth on PositionManager to

burn the underlying.

Inputs

- `_to`
 - **Control:** Fully caller-controlled; the function is permissionless.
 - **Constraints:** Must be non-zero.
 - **Impact:** Receives `_amount` of the newly minted single-condition combinatorial YES position.
- `_underlyingPositionId`
 - **Control:** Fully caller-controlled.
 - **Constraints:** `outcomeIndex()` must be 0 or 1. `moduleId()` must be BINARY or NEGRISK. The module must hold `>= _amount`.
 - **Impact:** Stored as the sole leg of the new single-condition combinatorial condition; also determines the input position to burn.
- `_amount`
 - **Control:** Fully caller-controlled.
 - **Constraints:** Module must hold `>= _amount` of `_underlyingPositionId`.
 - **Impact:** Quantity of combinatorial YES minted and of underlying burned.

Branches and code coverage

Intended branches

- underlying position wrapped into single-condition combinatorial YES position
 - ☒ Test coverage
- wrap then unwrap roundtrip
 - ☒ Test coverage

Negative behavior

- `outcomeIndex >= 2` reverts `InvalidOutcomeIndex`
 - ☐ Negative test
- underlying from unsupported module reverts `InvalidConditionModule`
 - ☐ Negative test

Function call analysis

- CombinatorialModule.wrap ->
CombinatorialModule._validateConditionModule(_underlyingPositionId)
 - **What is controllable?** _underlyingPositionId.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Pure check with no return value; reverts InvalidConditionModule if moduleId is not BINARY or NEGRISK.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Pure function; no external calls, no reentrancy, no side effects.
- CombinatorialModule.wrap ->
CombinatorialModule._storeLegsFromMemory([_underlyingPositionId])
 - **What is controllable?** _underlyingPositionId.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns the ConditionId for the single-leg set.
 - **What happens if it reverts, reenters or does other unusual control flow?**
There are no problems here.
- CombinatorialModule.wrap -> PositionManager.mint(_to, combinatorialPositionId, _amount)
 - **What is controllable?** _to and _amount.
 - **If the return value is controllable, how is it used and how can it go wrong?**
No return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Trusted in-system call. Reverts on zero _to or overflow.
- CombinatorialModule.wrap -> PositionManager.burn(_underlyingPositionId, _amount)
 - **What is controllable?** _underlyingPositionId and _amount.
 - **If the return value is controllable, how is it used and how can it go wrong?**
No return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Trusted in-system call; no reentrancy. Reverts if the module's balance of _underlyingPositionId is below _amount.

5.5. Module: OracleAggregator.sol

Function: disputeResult(bytes32 _requestId)

Called by a registered disputer module to challenge the currently proposed outcome for a request. A dispute simply increments state.disputeCount. Once that count meets cfg.disputerThreshold, the status transitions to ArbitrationRequested and IArbitratorModule.onArbitrationTriggered is invoked with the disputed proposedResultHash. There is no per-address deduplication: the same disputer module address may call this multiple times, each increment counting toward the threshold.

Inputs

- `_requestId`
 - **Control:** Fully caller-controlled.
 - **Constraints:** EventId for the provided requestId must be registered.
 - **Impact:** The id of the event or condition to dispute.

Branches and code coverage

Intended branches

- Reaching `disputerThreshold` triggers arbitration and arbitrator hook
 - ☒ Test coverage
- Same module disputing twice reaches threshold of 2
 - ☒ Test coverage
- Two different disputer modules increment `disputeCount`
 - ☒ Test coverage

Negative behavior

- Reverts when aggregator is paused (`whenUnpaused`)
 - ☒ Negative test
- Reverts when request not Active (`RequestNotActive`)
 - ☐ Negative test
- Reverts when no proposal to challenge (`NoProposalToChallenge`)
 - ☐ Negative test
- Reverts when dispute window expired (`DisputeWindowExpired`)
 - ☐ Negative test
- Reverts when caller is not a registered disputer module (`NotRegisteredModule`)
 - ☐ Negative test

Function call analysis

- `OracleAggregator.disputeResult` -> `IArbitratorModule.onArbitrationTriggered` (external module call)
 - **What is controllable?** `_requestId`.

- **If the return value is controllable, how is it used and how can it go wrong?**
No return value.
- **What happens if it reverts, reenters or does other unusual control flow?**
There are no problems here.

Function: `finalize(bytes32 _requestId, uint256[] calldata _result)`

Permissionless finalization path: callable by anyone once the dispute window has elapsed and a proposal exists. Validates that the caller-controlled `_result` matches the previously stored `proposedResultHash`, then delegates to `_finalizeConditions` to stamp Resolved and forward each sub-condition payout to `cfg.targetContract`. Unlike `resolveResult`, there is no privileged caller requirement because the security guarantee is the hash check: only the exact payout vector that accumulated enough reporter votes can be passed.

Inputs

- `_requestId`
 - **Control:** Fully caller-controlled.
 - **Constraints:** EventId for the provided requestId must be registered.
 - **Impact:** The id of the event or condition to finalize.
- `_result`
 - **Control:** Fully caller-controlled.
 - **Constraints:** Length must equal `cfg.resultLength`; sum of all elements must be $\leq \text{OptimisticOraclePayoutLib.RESULT_DENOMINATOR}$.
 - **Impact:** Determines the per-condition results forwarded to `cfg.targetContract`. The hash check means the caller cannot deviate from what reporters voted on.

Branches and code coverage

Intended branches

- Finalizes incremental negrisk and reports translated binary payout to target
 - ☒ Test coverage
- Target already resolved with matching payout treated as success
 - ☒ Test coverage

Negative behavior

- Reverts when aggregator is paused (`whenUnpaused`)

- ☒ Negative test
 - Target payout mismatch bubbles revert and leaves request Active
- ☒ Negative test
 - Target reportResult revert bubbles up and request stays Active
- ☒ Negative test
 - Reverts when request not Active (RequestNotActive)
- ☐ Negative test
 - Reverts when no proposal exists (ThresholdNotMet)
- ☐ Negative test
 - Reverts when dispute window still active (DisputeWindowActive)
- ☐ Negative test
 - Reverts when result does not match proposed hash (InvalidResultHash)
- ☐ Negative test

Function call analysis

- OracleAggregator.finalize -> OracleAggregator._isActiveStatus
 - **What is controllable?** Nothing is controllable.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Not controllable externally. Returns false for Resolved or ArbitrationRequested, preventing permissionless finalization once arbitration is active or the request is already settled.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Pure function; cannot revert or reenter.
- OracleAggregator.finalize -> OracleAggregator._finalizeConditions -> IBinaryReporter.reportResult (external target call, try/catch)
 - **What is controllable?** _result values are caller-controlled but constrained by the hash check immediately before _finalizeConditions is called.
 - **If the return value is controllable, how is it used and how can it go wrong?**
No return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
ConditionAlreadyResolved errors from the target are silently skipped. All other errors are revert.

Function: initializeRequest(InitParams calldata _params)

This function allows operator role to register a new oracle resolution configuration for an event, binding a RequestConfig to the provided eventId and registering all reporter and disputer

modules. After registration, the request becomes immediately Active.

Inputs

- `_params:eventId`
 - **Control:** Operator-controlled.
 - **Constraints:** Must not already have a registered RequestConfig. For ATOMIC_NEGRISK, its encoded `arity()` bits must equal `_params.resultLength`.
 - **Impact:** Determines the id of the event that will be registered.
- `_params:marketType`
 - **Control:** Operator-controlled.
 - **Constraints:** Enum `BINARY = 0`, `INCREMENTAL_NEGRISK = 1`, `ATOMIC_NEGRISK = 2`.
 - **Impact:** Determines the market type.
- `_params:targetContract`
 - **Control:** Operator-controlled.
 - **Constraints:** Must be non-zero.
 - **Impact:** The address that receives `reportResult(conditionId, result)` calls when conditions finalize.
- `_params:resultLength`
 - **Control:** Operator-controlled.
 - **Constraints:** Must be `== 1` for `BINARY/INCREMENTAL_NEGRISK`. Must equal `eventId.arity()` for `ATOMIC_NEGRISK`.
 - **Impact:** Later `reportResult` calls validate that submitted result arrays have this exact length. A misconfigured value permanently breaks reporting for this event.
- `_params:reporterModules`
 - **Control:** Operator-controlled array.
 - **Constraints:** No validation that the array is non-empty (a request with zero reporters would be permanently unreportable). No zero-address check on individual module fields. There is no duplicates check; however, in the current module implementation there is a check that `eventId.asCondition()` is already initialized, so in the case of duplicates it will revert.
 - **Impact:** Each address is registered via `isReporterModule[eventId][module] = true`. A zero-address module entry would register `address(0)` as a reporter. The `reporterThreshold` cannot be met without enough registered modules.
- `_params:reporterModules:module`
 - **Control:** Operator-controlled array.

- **Constraints:** No validation that the array is non-empty (a request with zero reporters would be permanently unreportable). No zero-address check on individual module fields.
 - **Impact:** Each address is registered via `isReporterModule[eventId][module] = true`. A zero-address module entry would register `address(0)` as a reporter. The `reporterThreshold` cannot be met without enough registered modules. Becomes permanently authorized to call `reportResult` on this aggregator for this event.
- `_params:reporterModules:initData`
 - **Control:** Operator-controlled arbitrary bytes.
 - **Constraints:** If `length > 0`, `IRReporterModule(module).initializeReporterModule(eventId, initData)` is called.
 - **Impact:** Passed to the module's initializer. Incorrectly configured module could revert here, blocking the entire `initializeRequest` call.
- `_params:reporterThreshold`
 - **Control:** Operator-controlled.
 - **Constraints:** Must be ≥ 1 . No upper bound validation.
 - **Impact:** The number of distinct reporter votes required to propose an outcome.
- `_params:disputerModules`
 - **Control:** Operator-controlled array.
 - **Constraints:** There is no constraint. An empty array means no dispute mechanism is available, making any proposal finalizable after the liveness window without challenge.
 - **Impact:** Each address is registered via `isDisputerModule[eventId][module] = true` to be able to dispute proposals for this event.
- `_params:disputerModules:module`
 - **Control:** Operator-controlled.
 - **Constraints:** No constraints.
 - **Impact:** Allows module to dispute proposals for this event.
- `_params:disputerModules:initData`
 - **Control:** Operator-controlled arbitrary bytes.
 - **Constraints:** If `length > 0`, triggers external call to disputer module initializer.
 - **Impact:** Revert aborts the entire initialization.
- `_params:disputerThreshold`
 - **Control:** Operator-controlled.
 - **Constraints:** When `disputerModules.length > 0`; must be ≥ 1 .

- **Impact:** The number of disputes required to trigger arbitration.
- `_params:arbiterModule`
 - **Control:** Operator-controlled address.
 - **Constraints:** When `disputerModules.length > 0`, must be non-zero.
 - **Impact:** The address authorized to call `finalizeArbitration`.
- `_params:arbiterInitData`
 - **Control:** Operator-controlled.
 - **Constraints:** If `length > 0`, triggers external call.
 - **Impact:** Configures the arbiter.
- `_params:livenessWindow`
 - **Control:** Operator-controlled.
 - **Constraints:** When `disputerModules.length > 0`; must be `> 0`.
 - **Impact:** Determines how long after a proposal is made the dispute window stays open.

Branches and code coverage

Intended branches

- Binary request stored and becomes Active
 - ☒ Test coverage
- Request with no disputer modules stored without dispute config
 - ☒ Test coverage
- Atomic negrisk request with `resultLength == arity` stored
 - ☐ Test coverage
- Arbiter init data triggers `initializeArbiterModule`
 - ☐ Test coverage

Negative behavior

- Reverts for non-operator caller (`onlyOperator`)
 - ☒ Negative test
- Reverts when request already exists for `eventId`
 - ☒ Negative test
- Reverts when atomic `resultLength != arity` (`InvalidResultLength`)
 - ☒ Negative test
- Reverts on zero `targetContract` (`InvalidConfig`)

- ☒ Negative test
 - Reverts on zero reporterThreshold (InvalidConfig)
- ☒ Negative test
 - Reverts on zero disputerThreshold with disputers (InvalidConfig)
- ☒ Negative test
 - Reverts on zero arbitratorModule with disputers (InvalidConfig)
- ☒ Negative test
 - Reverts on zero livenessWindow with disputers (InvalidConfig)
- ☒ Negative test
 - Reverts on non-canonical eventId (EventIdLib.from)
- ☒ Negative test

Function call analysis

- OracleAggregator.initializeRequest ->
 OracleAggregator._registerReporterModules(_eventId, _reporters) ->
 IReporterModule(module).initializeReporterModule(_eventId, initData)
 - **What is controllable?** _params.reporterModules, _eventId, initData.
 - **If the return value is controllable, how is it used and how can it go wrong?**
No return value used.
 - **What happens if it reverts, reenters or does other unusual control flow?**
External call to an operator-configured module. For the ChainlinkReporterModule, reverts if the startTime + duration already in the past.
- OracleAggregator.initializeRequest ->
 OracleAggregator._registerDisputerModules(_eventId, _disputers) ->
 IDisputerModule(module).initializeDisputerModule(_eventId, initData)
 - **What is controllable?** _params.disputerModules, _eventId, initData.
 - **If the return value is controllable, how is it used and how can it go wrong?**
No return value used.
 - **What happens if it reverts, reenters or does other unusual control flow?** No problems here.
- OracleAggregator.initializeRequest ->
 IArbitratorModule(_params.arbitratorModule).initializeArbitratorModule(_eventId, _arbitratorInitData)
 - **What is controllable?** _params.arbitratorModule, _params.arbitratorInitData, _eventId.
 - **If the return value is controllable, how is it used and how can it go wrong?**
No return value used.

- **What happens if it reverts, reenters or does other unusual control flow?**
There are no problems here.

Function: `reportResult(bytes32 _requestId, uint256[] calldata _result)`

Called by a registered reporter module to submit a result vector for a given request. Each caller may vote exactly once per `_requestId`. Votes are counted by the hash of `_result`; once the `reporterThreshold` is reached for a particular result hash and no outcome has been proposed yet, the outcome is recorded and the dispute window opens. Subsequent threshold-crossings for the same request are silently ignored.

Inputs

- `_requestId`
 - **Control:** Fully caller-controlled.
 - **Constraints:** EventId for the provided requestId must be registered.
 - **Impact:** The id of the event or condition to report on.
- `_result`
 - **Control:** Fully caller-controlled.
 - **Constraints:** Length must equal `cfg.resultLength`; the sum of all elements must be $\leq \text{OptimisticOraclePayoutLib.RESULT_DENOMINATOR}$.
 - **Impact:** Only the first hash to reach `reporterThreshold` is recorded as the proposed outcome. Other hashes are ignored.

Branches and code coverage

Intended branches

- Two distinct modules increment vote count for same request
 - ☒ Test coverage
- Same module votes on different subconditions independently
 - ☒ Test coverage
- Reaching `reporterThreshold` proposes outcome and opens dispute window
 - ☒ Test coverage

Negative behavior

- Reverts when aggregator is paused (`whenUnpaused`)
 - ☒ Negative test
- Reverts when no request found (`RequestNotFound`)

- ☒ Negative test
 - Reverts when same module votes twice (AlreadyVoted)
- ☒ Negative test
 - Reverts on out-of-range conditionIndex (InvalidConditionIndex)
- ☒ Negative test
 - Reverts when atomic request given child conditionId (InvalidRequestId)
- ☒ Negative test
 - Reverts on wrong result length (InvalidResultLength)
- ☒ Negative test
 - Reverts when caller is not a registered reporter module (NotRegisteredModule)
- ☐ Negative test
 - Reverts when request is not Active (RequestNotActive)
- ☐ Negative test
 -

Function: resolveResult(bytes32 _requestId, uint256[] calldata _result)

Called by a registered arbitrator module or an admin (`_ROLE_0`) to resolve a request. If the request is already Resolved, it returns early as a no-op. After verifying caller authorization and validating `_result`, it optionally notifies the arbitrator via `onArbitrationResolved`. Unlike `reportResult`, no cross-check is made against any previously proposed hash — the authorized caller sets the final result directly.

Inputs

- `_requestId`
 - **Control:** Fully caller-controlled.
 - **Constraints:** EventId for the provided requestId must be registered.
 - **Impact:** The id of the event or condition to resolve.
- `_result`
 - **Control:** Fully caller-controlled.
 - **Constraints:** Length must equal `cfg.resultLength`; sum of all elements must be $\leq \text{OptimisticOraclePayoutLib.RESULT_DENOMINATOR}$.
 - **Impact:** Directly sets `state.proposedResultHash` and the per-condition payouts forwarded to `cfg.targetContract`. There is no comparison against any previously voted hash; as a result, a privileged caller can resolve with any valid result regardless of what reporters proposed.

Branches and code coverage

Intended branches

- Admin resolves atomic negrisk, reporting all subconditions to target
 - ☒ Test coverage
- Admin override during arbitration clears arbitrator state via hook
 - ☒ Test coverage
- Admin override outside arbitration skips arbitrator hook
 - ☒ Test coverage
- Reverting arbitrator hook is caught, emits ArbitratorHookFailed and still resolves
 - ☒ Test coverage
- No-op when request already Resolved
 - ☒ Test coverage

Negative behavior

- Reverts when aggregator is paused (whenUnpaused)
 - ☒ Negative test
- Reverts for caller that is neither arbitrator nor admin (NotAuthorized)
 - ☒ Negative test

Function call analysis

- OracleAggregator.resolveResult -> OracleAggregator._validateResult
 - **What is controllable?** _result is fully caller-controlled.
 - **If the return value is controllable, how is it used and how can it go wrong?**
No return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Reverts if the length or sum of the result is invalid.
- OracleAggregator.resolveResult -> IArbitratorModule.onArbitrationResolved (external module call, try/catch)
 - **What is controllable?** _requestId.
 - **If the return value is controllable, how is it used and how can it go wrong?**
No return value. The hook is best-effort: any revert is swallowed and emits ArbitratorHookFailed, then resolution proceeds. A reverting arbitrator cannot block an admin override.
 - **What happens if it reverts, reenters or does other unusual control flow?**
There are no problems here.

- OracleAggregator.resolveResult -> OracleAggregator._finalizeConditions -> IBinaryReporter.reportResult (external target call, try/catch)
 - **What is controllable?** `_result` values are caller-controlled.
 - **If the return value is controllable, how is it used and how can it go wrong?**
No return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
status is set to Resolved at the top of `_finalizeConditions`, before the reporting loop. ConditionAlreadyResolved errors from the target are silently skipped; all other errors revert.

6. Assessment Results

During our assessment on the scoped Polymarket V2 contracts, we discovered 13 findings. No critical issues were found. Three findings were of medium impact, five were of low impact, and the remaining findings were informational in nature.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.